

```

1 // Comments marked by initials ER, KP, KL, DS, EC
2 //#####
3 // FILE:    FinalProjectStarter_main.c
4 //
5 // TITLE:   Final Project Starter
6 //#####
7
8 // Included Files
9 #include <stdio.h>
10 #include <stdlib.h>
11 #include <stdarg.h>
12 #include <string.h>
13 #include <math.h>
14 #include <limits.h>
15 #include "F28x_Project.h"
16 #include "F2837xD_SWPrioritizedIsrLevels.h"
17 #include "driverLib.h"
18 #include "device.h"
19 #include "F28379dSerial.h"
20 #include "song.h"
21 #include "dsp.h"
22 #include "fpu32/fpu_rfft.h"
23 #include "xy.h"
24 #include "MatrixMath.h"
25 #include "SE423Lib.h"
26 #include "OptiTrack.h"
27
28 #define PI          3.1415926535897932384626433832795
29 #define TWOPI       6.283185307179586476925286766559
30 #define HALFPI      1.5707963267948966192313216916398
31 // The Launchpad's CPU Frequency set to 200 you should not change this value
32 #define LAUNCHPAD_CPU_FREQUENCY 200
33
34 // ----- code for CAN start here -----
35 #include "F28379dCAN.h"
36 // #define TX_MSG_DATA_LENGTH      4
37 // #define TX_MSG_OBJ_ID           0 //transmit
38
39 #define RX_MSG_DATA_LENGTH      8
40 #define RX_MSG_OBJ_ID_1         1 //measurement from sensor 1
41 #define RX_MSG_OBJ_ID_2         2 //measurement from sensor 2
42 #define RX_MSG_OBJ_ID_3         3 //quality from sensor 1
43 #define RX_MSG_OBJ_ID_4         4 //quality from sensor 2
44 // ----- code for CAN end here -----
45
46 // Interrupt Service Routines predefinition
47 __interrupt void cpu_timer0_isr(void);
48 __interrupt void cpu_timer1_isr(void);
49 __interrupt void cpu_timer2_isr(void);
50 __interrupt void SWI1_HighestPriority(void);
51 __interrupt void SWI2_MiddlePriority(void);
52 __interrupt void SWI3_LowestPriority(void);
53 __interrupt void ADCC_ISR(void);
54 __interrupt void SPIB_isr(void);
55 // ----- code for CAN start here -----
56 __interrupt void can_isr(void);
57 // ----- code for CAN end here -----
58
59 void setF28027EPWM1A(float controleffort);
60 int16_t EPwm1A_F28027 = 1500;
61 void setF28027EPWM2A(float controleffort);
62 int16_t EPwm2A_F28027 = 1500;
63
64
65 // ----- code for CAN start here -----
66 // volatile uint32_t txMsgCount = 0;
67 // extern uint16_t txMsgData[4];
68
69 volatile uint32_t rxMsgCount_1 = 0;
70 volatile uint32_t rxMsgCount_2 = 0;
71 extern uint16_t rxMsgData[8];
72
73 uint32_t dis_raw_1[2];
74 uint32_t dis_raw_2[2];
75 uint32_t dis_1 = 0;
76 uint32_t dis_2 = 0;
77
78 uint32_t quality_raw_1[4];
79 uint32_t quality_raw_2[4];
80 float quality_1 = 0.0;
81 float quality_2 = 0.0;
82
83 uint32_t lightlevel_raw_1[4];
84 uint32_t lightlevel_raw_2[4];
85 float lightlevel_1 = 0.0;
86 float lightlevel_2 = 0.0;
87
88 uint32_t measure_status_1 = 0;
89 uint32_t measure_status_2 = 0;
90
91 volatile uint32_t errorFlag = 0;
92 // ----- code for CAN end here -----
93
94 uint32_t numTimer0calls = 0;
95 uint16_t UARTPrint = 0;
96
97 float printLV1 = 0;
98 float printLV2 = 0;
99
100 float printLinux1 = 0;
101 float printLinux2 = 0;
102
103 uint16_t LADARi = 0;
104 char G_command[] = "G04472503\n"; //command for getting distance -120 to 120 degree
105 uint16_t G_len = 11; //length of command
106 xy ladar_pts[228]; //xy data
107 float LADARrightfront = 0;
108 float LADARfront = 0;
109 float LADARtemp_x = 0;
110 float LADARtemp_y = 0;
111 extern datapts ladar_data[228];
112
113 extern uint16_t newLinuxCommands;
114 extern float LinuxCommands[CMDNUM_FROM_FLOATS];
115
116 extern uint16_t NewLVData;
117 extern float fromLVvalues[LVNUM_TOFROM_FLOATS];
118 extern LVSendFloats_t DataToLabView;
119 extern char LVsenddata[LVNUM_TOFROM_FLOATS*4+2];
120 extern uint16_t LADARpingpong;
121 extern uint16_t NewCAMdataThreshold1; // Flag new data
122 extern float fromCAMvaluesThreshold1[CAMNUM_FROM_FLOATS];
123 extern uint16_t NewCAMdataThreshold2; // Flag new data
124 extern float fromCAMvaluesThreshold2[CAMNUM_FROM_FLOATS];
125 extern uint16_t NewCAMdataThreshold3; // Flag new data
126 extern float fromCAMvaluesThreshold3[CAMNUM_FROM_FLOATS];
127
128 extern uint16_t NewCAMdataAprilTag1; // Flag new data
129 extern float fromCAMvaluesAprilTag1[CAMNUM_FROM_FLOATS];
130
131 float tagid = 0;
132 float tagx = 0;
133 float tagy = 0;
134 float tagz = 0;
135 float tagthetax = 0;
136 float tagthetay = 0;
137 float tagthetaz = 0;
138 int32_t numtag = 0;
139 int32_t AprilTagState = 10;
140 float KpAprilTag = -1.0;

```

```

141 float AprilTagSetPoint = 0;
142 int32_t AprilTagCount = 0;
143
144 float MaxAreaThreshold1 = 0;
145 float MaxColThreshold1 = 0;
146 float MaxRowThreshold1 = 0;
147 float NextLargestAreaThreshold1 = 0;
148 float NextLargestColThreshold1 = 0;
149 float NextLargestRowThreshold1 = 0;
150 float NextNextLargestAreaThreshold1 = 0;
151 float NextNextLargestColThreshold1 = 0;
152 float NextNextLargestRowThreshold1 = 0;
153
154 float MaxAreaThreshold2 = 0;
155 float MaxColThreshold2 = 0;
156 float MaxRowThreshold2 = 0;
157 float NextLargestAreaThreshold2 = 0;
158 float NextLargestColThreshold2 = 0;
159 float NextLargestRowThreshold2 = 0;
160 float NextNextLargestAreaThreshold2 = 0;
161 float NextNextLargestColThreshold2 = 0;
162 float NextNextLargestRowThreshold2 = 0;
163
164 //EC
165 float MaxAreaThreshold3 = 0;
166 float MaxColThreshold3 = 0;
167 float MaxRowThreshold3 = 0;
168 float NextLargestAreaThreshold3 = 0;
169 float NextLargestColThreshold3 = 0;
170 float NextLargestRowThreshold3 = 0;
171 float NextNextLargestAreaThreshold3 = 0;
172 float NextNextLargestColThreshold3 = 0;
173 float NextNextLargestRowThreshold3 = 0;
174
175 uint32_t numThres1 = 0;
176 uint32_t numThres2 = 0;
177 uint32_t numThres3 = 0; //EC
178
179 pose ROBOTps = {0,0,0}; //robot position
180 pose LADARps = {3.5/12,0,0,1}; // 3.5/12 for front mounting, theta is not used in this current code
181 float LADARxoffset = 0;
182 float LADARyoffset = 0;
183
184 uint32_t timecount = 0;
185 int16_t RobotState = 1;
186 int16_t checkfronttally = 0;
187 int32_t WallFollowtime = 0;
188
189 #define NUMWAYPOINTS 10 //Modified NUMWAYPOINTS from 8 to 10 for two extra waypoint targets for the robot path - DS
190 uint16_t statePos = 0;
191 pose robotdest[NUMWAYPOINTS]; // array of waypoints for the robot
192 uint16_t i = 0; //for loop
193
194 uint16_t right_wall_follow_state = 2; // right follow
195 float Kp_front_wall = -2.0;
196 float front_turn_velocity = 0.2;
197 float left_turn_Stop_threshold = 3.5;
198 float Kp_right_wal = -4.0;
199 float ref_right_wall = 1.1;
200 float foward_velocity = 1.0;
201 float left_turn_Start_threshold = 1.3;
202 float turn_saturation = 2.5;
203
204 //RC Servo
205 float Gate_0 = -67.74;
206 float Gate_C = 0.40;
207 float RTong = -49.34; //KLEC: ball goes to the left
208 float MTong = -11.49;
209 float LTong = 1.47; //KLEC: ball goes to the right
210
211 float x_pred[3][1] = {{0},{0},{0}}; // predicted state
212
213 //more kalman vars
214 float B[3][2] = {{1,0},{1,0},{0,1}}; // control input model
215 float u[2][1] = {{0},{0}}; // control input in terms of velocity and angular velocity
216 float Bu[3][1] = {{0},{0},{0}}; // matrix multiplication of B and u
217 float z[3][1]; // state measurement
218 float eye3[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // 3x3 identity matrix
219 float K[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // optimal Kalman gain
220 #define ProcUncert 0.0001
221 #define CovScalar 10
222 float Q[3][3] = {{ProcUncert,0,ProcUncert/CovScalar},
223 {0,ProcUncert,ProcUncert/CovScalar},
224 {ProcUncert/CovScalar,ProcUncert/CovScalar,ProcUncert}}; // process noise (covariance of encoders and gyro)
225 #define MeasUncert 1
226 float R[3][3] = {{MeasUncert,0,MeasUncert/CovScalar},
227 {0,MeasUncert,MeasUncert/CovScalar},
228 {MeasUncert/CovScalar,MeasUncert/CovScalar,MeasUncert}}; // measurement noise (covariance of OptiTrack Motion Capture measurement)
229 float S[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance
230 float S_inv[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance matrix inverse
231 float P_pred[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // predicted covariance (measure of uncertainty for current position)
232 float temp_3x3[3][3]; // intermediate storage
233 float temp_3x1[3][1]; // intermediate storage
234 float ytilde[3][1]; // difference between predictions
235
236 // Optitrack Variables
237 int32_t OptiNumUpdatesProcessed = 0;
238 pose OPTITRACKps;
239 extern uint16_t new_optitrack;
240 extern float Optitrackdata[OPTITRACKDATASIZE];
241 int16_t newOPTITRACKpose=0;
242
243 int16_t adcc2result = 0;
244 int16_t adcc3result = 0;
245 int16_t adcc4result = 0;
246 int16_t adcc5result = 0;
247 float adcC2Volt = 0.0;
248 float adcC3Volt = 0.0;
249 float adcC4Volt = 0.0;
250 float adcC5Volt = 0.0;
251 int32_t numADCCalls = 0;
252 float LeftWheel = 0;
253 float RightWheel = 0;
254 float LeftWheel_1 = 0;
255 float RightWheel_1 = 0;
256 float LeftVel = 0;
257 float RightVel = 0;
258 float uLeft = 0;
259 float uRight = 0;
260 float HandValue = 0;
261 float vref = 0;
262 float turn = 0;
263 float gyro9250_drift = 0;
264 float gyro9250_angle = 0;
265 float old_gyro9250 = 0;
266 float gyroLPR510_angle = 0;
267 float gyroLPR510_offset = 0;
268 float gyroLPR510_drift = 0;
269 float old_gyroLPR510 = 0;
270 float gyro9250_radians = 0;
271 float gyroLPR510_radians = 0;
272
273 int16_t readdata[25];
274 int16_t IMU_data[9];
275
276 float accelx = 0;
277 float accely = 0;
278 float accelz = 0;
279 float gyro_x = 0;
280 float gyro_y = 0;
281 float gyro_z = 0;
282

```

```

283 // Needed global Variables
284 float accelx_offset = 0;
285 float accely_offset = 0;
286 float accelz_offset = 0;
287 float gyrox_offset = 0;
288 float gyroy_offset = 0;
289 float gyroz_offset = 0;
290 int16_t calibration_state = 0;
291 int32_t calibration_count = 0;
292 int16_t doneCal = 0;
293
294 #define MPU9250 1
295 #define DAN28027 2
296 int16_t CurrentChip = MPU9250;
297 int16_t DAN28027Garbage = 0;
298 int16_t dan28027adc1 = 0;
299 int16_t dan28027adc2 = 0;
300 uint16_t MPU9250IgnoreCNT = 0; //This is ignoring the first few interrupts if ADCC_ISR and start sending to IMU after these first few interrupts.
301 //Added variables to store camera-based ball distances - DS
302 float robotToBall2 = 0;
303 float robotToBall1 = 0;
304 float robotToBall3 = 0;
305
306 // angle of the hand encoder - ER
307 float RAngle = 0.0;
308 float colcentroid1 = 0.0;
309 float colcentroid2 = 0.0;
310 float colcentroid3 = 0.0;
311 float kpvision = -0.07; // vision proportional gain chosen so robot turns toward the ball centroid without overcorrecting - DS
312
313 int16_t count = 0;
314 int16_t statecount = 2000;
315 int16_t dwell=0;
316 float current= 0.0;
317 float target = 0.0;
318
319 void main(void)
320 {
321     // PLL, WatchDog, enable Peripheral Clocks
322     // This example function is found in the F2837xD_SysCtrl.c file.
323     InitSysCtrl();
324
325     InitGpio();
326
327     InitSE423DefaultGPIO();
328
329     //Scope for Timing
330     GPIO_SetupPinMux(11, GPIO_MUX_CPU1, 0);
331     GPIO_SetupPinOptions(11, GPIO_OUTPUT, GPIO_PUSH_PULL);
332     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
333
334     GPIO_SetupPinMux(32, GPIO_MUX_CPU1, 0);
335     GPIO_SetupPinOptions(32, GPIO_OUTPUT, GPIO_PUSH_PULL);
336     GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
337
338     GPIO_SetupPinMux(52, GPIO_MUX_CPU1, 0);
339     GPIO_SetupPinOptions(52, GPIO_OUTPUT, GPIO_PUSH_PULL);
340     GpioDataRegs.GPBCLEAR.bit.GPIO52 = 1;
341
342     GPIO_SetupPinMux(60, GPIO_MUX_CPU1, 0);
343     GPIO_SetupPinOptions(60, GPIO_OUTPUT, GPIO_PUSH_PULL);
344     GpioDataRegs.GPBCLEAR.bit.GPIO60 = 1;
345
346     GPIO_SetupPinMux(61, GPIO_MUX_CPU1, 0);
347     GPIO_SetupPinOptions(61, GPIO_OUTPUT, GPIO_PUSH_PULL);
348     GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
349
350     GPIO_SetupPinMux(67, GPIO_MUX_CPU1, 0);
351     GPIO_SetupPinOptions(67, GPIO_OUTPUT, GPIO_PUSH_PULL);
352     GpioDataRegs.GPCCLEAR.bit.GPIO67 = 1;
353
354     // ----- code for CAN start here -----
355     //GPIO17 - CANRXB
356     GPIO_SetupPinMux(17, GPIO_MUX_CPU1, 2);
357     GPIO_SetupPinOptions(17, GPIO_INPUT, GPIO_ASYNC);
358
359     //GPIO12 - CANTXB
360     GPIO_SetupPinMux(12, GPIO_MUX_CPU1, 2);
361     GPIO_SetupPinOptions(12, GPIO_OUTPUT, GPIO_PUSH_PULL);
362     // ----- code for CAN end here -----
363
364
365
366     // ----- code for CAN start here -----
367     // Initialize the CAN controller
368     InitCANB();
369
370     // Set up the CAN bus bit rate to 1000 kbps
371     setCANBitRate(20000000, 1000000);
372
373     // Enables Interrupt line 0, Error & Status Change interrupts in CAN_CTL register.
374     CanbRegs.CAN_CTL.bit.IE0 = 1;
375     CanbRegs.CAN_CTL.bit.EIE = 1;
376     // ----- code for CAN end here -----
377
378     // Clear all interrupts and initialize PIE vector table:
379     // Disable CPU interrupts
380     DINT;
381
382     // Initialize the PIE control registers to their default state.
383     // The default state is all PIE interrupts disabled and flags
384     // are cleared.
385     // This function is found in the F2837xD_PieCtrl.c file.
386     InitPieCtrl();
387
388     // Disable CPU interrupts and clear all CPU interrupt flags:
389     IER = 0x0000;
390     IFR = 0x0000;
391
392     // Initialize the PIE vector table with pointers to the shell Interrupt
393     // Service Routines (ISR).
394     // This will populate the entire table, even if the interrupt
395     // is not used in this example. This is useful for debug purposes.
396     // The shell ISR routines are found in F2837xD_DefaultIsr.c.
397     // This function is found in F2837xD_PieVect.c.
398     InitPieVectTable();
399
400     // Interrupts that are used in this example are re-mapped to
401     // ISR functions found within this project
402     EALLOW; // This is needed to write to EALLOW protected registers
403     PieVectTable.TIMER0_INT = &cpu_timer0_isr;
404     PieVectTable.TIMER1_INT = &cpu_timer1_isr;
405     PieVectTable.TIMER2_INT = &cpu_timer2_isr;
406     PieVectTable.SCIA_RX_INT = &RXAINT_recv_ready;
407     PieVectTable.SCIB_RX_INT = &RXBINT_recv_ready;
408     PieVectTable.SCIC_RX_INT = &RXCINT_recv_ready;
409     PieVectTable.SCID_RX_INT = &RXDINT_recv_ready;
410     PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
411     PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
412     PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
413     PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
414     PieVectTable.ADCC1_INT = &ADCC_ISR;
415     PieVectTable.SPIB_RX_INT = &SPIB_isr;
416     PieVectTable.EMIF_ERROR_INT = &SWI1_HighestPriority; // Using Interrupt12 interrupts that are not used as SWIs
417     PieVectTable.RAM_CORRECTABLE_ERROR_INT = &SWI2_MiddlePriority;
418     PieVectTable.FLASH_CORRECTABLE_ERROR_INT = &SWI3_LowestPriority;
419     // ----- code for CAN start here -----
420     PieVectTable.CANB0_INT = &can_isr;
421     // ----- code for CAN end here -----
422     EDIS; // This is needed to disable write to EALLOW protected registers
423
424     // Initialize the CpuTimers Device Peripheral. This function can be

```

```

425 // found in F2837xD_CpuTimers.c
426 InitCpuTimers();
427
428 // Configure CPU-Timer 0, 1, and 2 to interrupt every second:
429 // 200MHz CPU Freq, Period (in uSeconds)
430 ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000); // Currently not used for any purpose
431 ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 100000); // !!!!! Important, Used to command LADAR every 100ms. Do not Change.
432 ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 40000); // Currently not used for any purpose
433
434 // Enable CpuTimer Interrupt bit TIE
435 CpuTimer0Regs.TCR.all = 0x4000;
436 CpuTimer1Regs.TCR.all = 0x4000;
437 CpuTimer2Regs.TCR.all = 0x4000;
438
439 DELAY_US(1000000); // Delay 1 second giving Lidar Time to power on after system power on
440
441 init_serialSCIB(&SerialB,19200);
442 init_serialSCIC(&SerialC,19200);
443
444 for (LADARi = 0; LADARi < 228; LADARi++) {
445     Ladar_data[LADARi].angle = ((3*LADARi+44)*0.3515625-135)*0.01745329; //0.017453292519943 is pi/180, multiplication is faster; 0.3515625 is 360/1024
446 }
447
448 init_eQEPs();
449 init_EPWM1and2();
450 init_RCServoPWM_3AB_5AB_6A();
451 init_ADCsAndDACs();
452 setupSpib();
453
454 EALLOW;
455 EPwm4Regs.ETSEL.bit.SOCAEN = 0; // Disable SOC on A group
456 EPwm4Regs.TBCTL.bit.CTRMODE = 3; // freeze counter
457 EPwm4Regs.ETSEL.bit.SOCASEL = 2; // Select Event when counter equal to PRD
458 EPwm4Regs.ETPS.bit.SOCAPRD = 1; // Generate pulse on 1st event
459 EPwm4Regs.TBCTR = 0x0; // Clear counter
460 EPwm4Regs.TBPHS.bit.TBPHS = 0x0000; // Phase is 0
461 EPwm4Regs.TBCTL.bit.PHSEN = 0; // Disable phase loading
462 EPwm4Regs.TBCTL.bit.CLKDIV = 0; // divide by 1 50Mhz Clock
463 EPwm4Regs.TBPRD = 50000; // Set Period to 1ms sample. Input clock is 50MHz.
464 // Notice here that we are not setting CMPA or CMPB because we are not using the PWM signal
465 EPwm4Regs.ETSEL.bit.SOCAEN = 1; //enable SOCA
466 //EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, wait to do this right before enabling interrupts
467 EDIS;
468
469 // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
470 // which is connected to CPU-Timer 1, and CPU int 14, which is connected
471 // to CPU-Timer 2: int 12 is for the SWI.
472 IER |= M_INT1;
473 IER |= M_INT6;
474 IER |= M_INT8; // SCIC SCID
475 IER |= M_INT9; // SCIA
476 IER |= M_INT12;
477 IER |= M_INT13;
478 IER |= M_INT14;
479
480 // Enable TINT0 in the PIE: Group 1 interrupt 7
481 PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
482 PieCtrlRegs.PIEIER1.bit.INTx3 = 1;
483 PieCtrlRegs.PIEIER6.bit.INTx3 = 1;
484 PieCtrlRegs.PIEIER12.bit.INTx9 = 1; //SWI1
485 PieCtrlRegs.PIEIER12.bit.INTx10 = 1; //SWI2
486 PieCtrlRegs.PIEIER12.bit.INTx11 = 1; //SWI3 Lowest priority
487 // ----- code for CAN start here -----
488 // Enable CANB in the PIE: Group 9 interrupt 7
489 PieCtrlRegs.PIEIER9.bit.INTx7 = 1;
490 // ----- code for CAN end here -----
491
492 // ----- code for CAN start here -----
493 // Enable the CAN interrupt signal
494 CanbRegs.CAN_GLB_INT_EN.bit.GLBINT0_EN = 1;
495 // ----- code for CAN end here -----
496
497 // 58.5 in * 7.5 tiles = 438.75 inches = 11.14425 meters
498
499
500 robotdest[0].x = 0; robotdest[0].y = 7.5 *5;
501 robotdest[1].x = 0*5; robotdest[1].y = 0*5;
502 //middle of bottom
503 // robotdest[2].x = 0; robotdest[2].y = 2;
504 // //outside the course
505 // robotdest[3].x = 0; robotdest[3].y = -3;
506 // //back to middle
507 // robotdest[4].x = 0; robotdest[4].y = 2;
508 // robotdest[5].x = 4; robotdest[5].y = 2;
509 // robotdest[6].x = 4; robotdest[6].y = 10;
510 // robotdest[7].x = 0; robotdest[7].y = 9;
511 // // the two extra points that we added for the robot to go to after the initial course one in the middle of the field and one outside near the wall - ER
512 // robotdest[8].x = 2; robotdest[8].y = 5;
513 // robotdest[9].x = -2; robotdest[9].y = -1;
514
515 // ROBOTps will be updated by Optitrack during gyro calibration
516 // TODO: specify the starting position of the robot
517 ROBOTps.x = 0; //the estimate in array form (useful for matrix operations)
518 ROBOTps.y = 0;
519 ROBOTps.theta = 0; // was -PI: need to flip OT ground plane to fix this
520 x_pred[0][0] = ROBOTps.x; //estimate in structure form (useful elsewhere)
521 x_pred[1][0] = ROBOTps.y;
522 x_pred[2][0] = ROBOTps.theta;
523
524 AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
525 PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
526 SpibRegs.SPIFFRX.bit.RXFFOVCCLR=1; // Clear Overflow flag
527 SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
528 PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
529 EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, and enter up count mode
530
531 init_serialSCIA(&SerialA,115200);
532 init_serialSCID(&SerialD,2083332);
533
534 // Enable global Interrupts and higher priority real-time debug events
535 EINT; // Enable Global interrupt INTM
536 ERTM; // Enable Global realtime interrupt DBGM
537 // ----- code for CAN start here -----
538 // Measured Distance from 1
539 // Initialize the receive message object 1 used for receiving CAN messages.
540 // Message Object Parameters:
541 // Message Object ID Number: 1
542 // Message Identifier: 0x060b0101
543 // Message Frame: Standard
544 // Message Type: Receive
545 // Message ID Mask: 0x0
546 // Message Object Flags: Receive Interrupt
547 // Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
548 // for a Receive mailbox)
549 //
550 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_1, 0x060b0101, CAN_MSG_FRAME_EXT,
551 CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
552 RX_MSG_DATA_LENGTH);
553
554 // Measured Distance from 2
555 // Initialize the receive message object 2 used for receiving CAN messages.
556 // Message Object Parameters:
557 // Message Object ID Number: 2
558 // Message Identifier: 0x060b0102
559 // Message Frame: Standard
560 // Message Type: Receive
561 // Message ID Mask: 0x0
562 // Message Object Flags: Receive Interrupt
563 // Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
564 // for a Receive mailbox)
565 //
566

```

```

568 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_2, 0x060b0102, CAN_MSG_FRAME_EXT,
569                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
570                       RX_MSG_DATA_LENGTH);
571
572 // Measurement Quality from 1
573 // Initialize the receive message object 2 used for receiving CAN messages.
574 // Message Object Parameters:
575 //   Message Object ID Number: 3
576 //   Message Identifier: 0x060b0201
577 //   Message Frame: Standard
578 //   Message Type: Receive
579 //   Message ID Mask: 0x0
580 //   Message Object Flags: Receive Interrupt
581 //   Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
582 //   for a Receive mailbox)
583 //
584 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_3, 0x060b0201, CAN_MSG_FRAME_EXT,
585                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
586                       RX_MSG_DATA_LENGTH);
587
588 // Measurement Quality from 2
589 // Initialize the receive message object 2 used for receiving CAN messages.
590 // Message Object Parameters:
591 //   Message Object ID Number: 4
592 //   Message Identifier: 0x060b0202
593 //   Message Frame: Standard
594 //   Message Type: Receive
595 //   Message ID Mask: 0x0
596 //   Message Object Flags: Receive Interrupt
597 //   Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
598 //   for a Receive mailbox)
599 //
600 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_4, 0x060b0202, CAN_MSG_FRAME_EXT,
601                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
602                       RX_MSG_DATA_LENGTH);
603
604 //
605 // Start CAN module operations
606 //
607 CanbRegs.CAN_CTL.bit.Init = 0;
608 CanbRegs.CAN_CTL.bit.CCE = 0;
609
610
611
612 // ----- code for CAN end here -----
613 char S_command[19] = "S1152000124000\n"; //this change the baud rate to 115200
614 uint16_t S_len = 19;
615 serial_sendSCIC(&SerialC, S_command, S_len);
616
617
618 DELAY_US(1000000); // Delay letting Lidar change its Baud rate
619 init_serialSCIC(&SerialC, 115200);
620
621
622 // LED1 is GPIO22
623 GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;
624 // LED2 is GPIO94
625 GpioDataRegs.GPCLEAR.bit.GPIO94 = 1;
626 // LED3 is GPIO95
627 GpioDataRegs.GPCLEAR.bit.GPIO95 = 1;
628 // LED4 is GPIO97
629 GpioDataRegs.GPDCLR.bit.GPIO97 = 1;
630 // LED5 is GPIO111
631 GpioDataRegs.GPDCLR.bit.GPIO111 = 1;
632
633
634 // IDLE loop. Just sit and loop forever (optional):
635 while(1)
636 {
637     if (UARTPrint == 1) {
638         //UART_printfLine(1, "RCangle:%.2f", RCangle);
639         if (readbuttons() == 0) {
640             UART_printfLine(1, "RobotState: %d", RobotState);
641             UART_printfLine(1, "01A:%.0fc:%.0fr:%.0f", MaxAreaThreshold1, MaxColThreshold1, MaxRowThreshold1);
642             UART_printfLine(2, "P1A:%.0fc:%.0fr:%.0f", MaxAreaThreshold2, MaxColThreshold2, MaxRowThreshold2);
643             UART_printfLine(1, "x:%.2f:y:%.2f:a:%.2f", ROBOTps.x, ROBOTps.y, ROBOTps.theta);
644             UART_printfLine(2, "purple: %.2f", robotToBallL2);
645         } else if (readbuttons() == 1) {
646             // UART_printfLine(1, "01A:%.0fc:%.0fr:%.0f", MaxAreaThreshold1, MaxColThreshold1, MaxRowThreshold1);
647             // UART_printfLine(2, "P1A:%.0fc:%.0fr:%.0f", MaxAreaThreshold2, MaxColThreshold2, MaxRowThreshold2);
648             //UART_printfLine(1, "LV1:%.3f LV2:%.3f", printLV1, printLV2);
649             //UART_printfLine(2, "Ln1:%.3f Ln2:%.3f", printLinux1, printLinux2);
650         } else if (readbuttons() == 2) {
651             UART_printfLine(1, "02A:%.0fc:%.0fr:%.0f", NextLargestAreaThreshold1, NextLargestColThreshold1, NextLargestRowThreshold1);
652             UART_printfLine(2, "P2A:%.0fc:%.0fr:%.0f", NextLargestAreaThreshold2, NextLargestColThreshold2, NextLargestRowThreshold2);
653             // UART_printfLine(1, "%.2f %.2f", adcC2Volt, adcC3Volt);
654             // UART_printfLine(2, "%.2f %.2f", adcC4Volt, adcC5Volt);
655         } else if (readbuttons() == 4) {
656             UART_printfLine(1, "03A:%.0fc:%.0fr:%.0f", NextNextLargestAreaThreshold1, NextNextLargestColThreshold1, NextNextLargestRowThreshold1);
657             UART_printfLine(2, "P3A:%.0fc:%.0fr:%.0f", NextNextLargestAreaThreshold2, NextNextLargestColThreshold2, NextNextLargestRowThreshold2);
658             // UART_printfLine(1, "L:%.3f R:%.3f", LeftVel, RightVel);
659             // UART_printfLine(2, "uL:%.2f uR:%.2f", uLeft, uRight);
660         } else if (readbuttons() == 8) {
661             UART_printfLine(1, "020x%.2f y%.2f", ladar_pts[20].x, ladar_pts[20].y);
662             UART_printfLine(2, "150x%.2f y%.2f", ladar_pts[150].x, ladar_pts[150].y);
663         } else if (readbuttons() == 3) {
664             UART_printfLine(1, "Vrf:%.2f trn:%.2f", vref, turn);
665             UART_printfLine(2, "MPU:%.2f LPR:%.2f", gyro9250_radians, gyroLPRS10_radians);
666         } else if (readbuttons() == 5) {
667             UART_printfLine(1, "0x:%.2f:0y:%.2f:0a:%.2f", OPTITRACKps.x, OPTITRACKps.y, OPTITRACKps.theta);
668             UART_printfLine(2, "State: %d", RobotState, statePos);
669         } else if (readbuttons() == 6) {
670             UART_printfLine(1, "D1 %ld D2 %ld", dis_1, dis_2);
671             UART_printfLine(2, "St1 %ld St2 %ld", measure_status_1, measure_status_2);
672         } else if (readbuttons() == 7) {
673             UART_printfLine(1, "%.0f, %.1f, %.1f, %.1f", tagid, tagx, tagy, tagz);
674             UART_printfLine(2, "%.0f, %.1f, %.1f, %.1f", tagthetax, tagthetay, tagthetaz);
675         }
676     }
677     UARTPrint = 0;
678 }
679 }
680 }
681
682 _interrupt void SPIB_isr(void) {
683     uint16_t i;
684     GpioDataRegs.GPBSET.bit.GPIO32 = 1;
685
686     GpioDataRegs.GPCSET.bit.GPIO66 = 1; //Pull CS high as done R/W
687     GpioDataRegs.GPASET.bit.GPIO29 = 1; // Pull CS high for DAN28027
688
689     if (CurrentChip == MPU9250) {
690         for (i=0; i<8; i++) {
691             readdata[i] = SpibRegs.SPIRXBUF; // readdata[0] is garbage
692         }
693
694         PostSWI1(); // Manually cause the interrupt for the SWI1
695     } else if (CurrentChip == DAN28027) {
696         DAN28027Garbage = SpibRegs.SPIRXBUF;
697         dan28027adc1 = SpibRegs.SPIRXBUF;
698         dan28027adc2 = SpibRegs.SPIRXBUF;
699         CurrentChip = MPU9250;
700         SpibRegs.SPIFFCT.bit.TXDLY = 0;
701     }
702
703     SpibRegs.SPIFFRX.bit.RXFFOVFLCLR=1; // Clear Overflow flag
704     SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag

```

```

709 PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
710 GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
711 }
712 }
713
714 //adcd1 pie interrupt
715 __interrupt void ADCC_ISR (void)
716 {
717     GpioDataRegs.GPASET.bit.GPIO11 = 1;
718     adcc2result = AdccResultRegs.ADCRESULT0;
719     adcc3result = AdccResultRegs.ADCRESULT1;
720     adcc4result = AdccResultRegs.ADCRESULT2;
721     adcc5result = AdccResultRegs.ADCRESULT3;
722
723     // Here covert ADCIND0 to volts
724     adc2Volt = adcc2result*3.0/4095.0;
725     adc3Volt = adcc3result*3.0/4095.0;
726     adc4Volt = adcc4result*3.0/4095.0;
727     adc5Volt = adcc5result*3.0/4095.0;
728
729     if (MPU9250ignoreCNT >= 1) {
730         CurrentChip = MPU9250;
731         SpibRegs.SPIFFCT.bit.TXDLY = 0;
732         SpibRegs.SPIFFRX.bit.RXFIL = 8;
733
734         GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
735
736         SpibRegs.SPITXBUF = ((0x8000)|(0x3A00));
737         SpibRegs.SPITXBUF = 0;
738         SpibRegs.SPITXBUF = 0;
739         SpibRegs.SPITXBUF = 0;
740         SpibRegs.SPITXBUF = 0;
741         SpibRegs.SPITXBUF = 0;
742         SpibRegs.SPITXBUF = 0;
743         SpibRegs.SPITXBUF = 0;
744     } else {
745         MPU9250ignoreCNT++;
746     }
747
748     numADCCcalls++;
749     AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
750     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
751     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
752 }
753 }
754
755 // cpu_timer0_isr - CPU Timer0 ISR
756 __interrupt void cpu_timer0_isr(void)
757 {
758     CpuTimer0.InterruptCount++;
759
760     numTimer0calls++;
761
762     if ((numTimer0calls%5) == 0) {
763         // Blink LaunchPad Red LED
764         //GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1;
765     }
766
767     // Acknowledge this interrupt to receive more interrupts from group 1
768     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
769 }
770
771 // cpu_timer1_isr - CPU Timer1 ISR
772 __interrupt void cpu_timer1_isr(void)
773 {
774     serial_sendSCIC(&SerialC, G_command, G_len);
775
776     CpuTimer1.InterruptCount++;
777 }
778
779 // cpu_timer2_isr CPU Timer2 ISR
780 __interrupt void cpu_timer2_isr(void)
781 {
782     // Blink LaunchPad Blue LED
783     //GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;
784     PostSWI3(); //Added PostSWI3 for Lab 7 Ex.1 Step 1 - DS
785     CpuTimer2.InterruptCount++;
786
787     // if ((CpuTimer2.InterruptCount % 10) == 0) {
788     //     UARTPrint = 1;
789     // }
790 }
791 }
792
793 void setF28027EPWM1A(float controleffort){
794     if (controleffort < -10) {
795         controleffort = -10;
796     }
797     if (controleffort > 10) {
798         controleffort = 10;
799     }
800     float value = (controleffort+10)*3000.0/20.0;
801     EPwm1A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
802 }
803 void setF28027EPWM2A(float controleffort){
804     if (controleffort < -10) {
805         controleffort = -10;
806     }
807     if (controleffort > 10) {
808         controleffort = 10;
809     }
810     float value = (controleffort+10)*3000.0/20.0;
811     EPwm2A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
812 }
813
814 //
815 // Connected to PIEIER12_9 (use MINT12 and MG12_9 masks):
816 //
817 __interrupt void SWI1_HighestPriority(void) // EMIF_ERROR
818 {
819     GpioDataRegs.GPBSET.bit.GPIO61 = 1;
820     // Set interrupt priority:
821     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
822     IER |= M_INT12;
823     IER &= MINT12; // Set "global" priority
824     PieCtrlRegs.PIEIER12.all &= MG12_9; // Set "group" priority
825     PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
826     __asm(" NOP");
827     EINT;
828
829     //#####
830     IMU_data[0] = readdata[1];
831     IMU_data[1] = readdata[2];
832     IMU_data[2] = readdata[3];
833     IMU_data[3] = readdata[5];
834     IMU_data[4] = readdata[6];
835     IMU_data[5] = readdata[7];
836
837     accelx = (((float)(IMU_data[0]))*4.0/32767.0);
838     accely = (((float)(IMU_data[1]))*4.0/32767.0);
839     accelz = (((float)(IMU_data[2]))*4.0/32767.0);
840     gyrox = (((float)(IMU_data[3]))*250.0/32767.0);
841     gyroy = (((float)(IMU_data[4]))*250.0/32767.0);
842     gyroz = (((float)(IMU_data[5]))*250.0/32767.0);
843
844     if(calibration_state == 0){
845         calibration_count++;
846         if (calibration_count == 2000) {
847             calibration_state = 1;
848             calibration_count = 0;
849         }
850     } else if(calibration_state == 1){

```

```

851 accelx_offset+=accelx;
852 accely_offset+=accely;
853 accelz_offset+=accelz;
854 gyrox_offset=-gyrox;
855 gyroy_offset=-gyroy;
856 gyroz_offset=-gyroz;
857 gyroLPR510_offset+=adcC5Volt;
858 calibration_count++;
859 if (calibration_count == 2000) {
860     calibration_state = 2;
861     accelx_offset/=2000.0;
862     accely_offset/=2000.0;
863     accelz_offset/=2000.0;
864     gyrox_offset/=2000.0;
865     gyroy_offset/=2000.0;
866     gyroz_offset/=2000.0;
867     gyroLPR510_offset/=2000.0;
868     calibration_count = 0;
869 }
870 } else if(calibration_state == 2) {
871
872     gyroLPR510_drift += (((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005)/(2000);
873     old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
874
875     gyro9250_drift += (((gyroz-gyro_offset) + old_gyro9250)*.0005)/(2000);
876     old_gyro9250 = gyroz-gyro_offset;
877     calibration_count++;
878
879     if (calibration_count == 2000) {
880         calibration_state = 3;
881         calibration_count = 0;
882         doneCal = 1;
883         newOPTITRACKpose = 0;
884     }
885 } else if(calibration_state == 3){
886     accelx -=(accelx_offset);
887     accely -=(accely_offset);
888     accelz -=(accelz_offset);
889     gyrox = gyrox_offset;
890     gyroy = gyroy_offset;
891     gyroz = gyroz_offset;
892     LeftWheel = readEncLeft();
893     RightWheel = readEncRight();
894     HandValue = readEncWheel();
895
896     gyro9250_angle = gyro9250_angle + (gyroz + old_gyro9250)*.0005 - gyro9250_drift;
897     old_gyro9250 = gyroz;
898
899     gyroLPR510_angle = gyroLPR510_angle + ((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005 - gyroLPR510_drift;
900     old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
901
902     gyro9250_radians = (gyro9250_angle * (PI/180.0));
903     gyroLPR510_radians = gyroLPR510_angle * 400 * (PI/180.0);
904
905     LeftVel = (1.235/12.0)*(LeftWheel - LeftWheel_1)*1000;
906     RightVel = (1.235/12.0)*(RightWheel - RightWheel_1)*1000;
907
908     if (newLinuxCommands == 1) {
909         newLinuxCommands = 0;
910         printLinux1 = LinuxCommands[0];
911         printLinux2 = LinuxCommands[1];
912         //?? = LinuxCommands[2];
913         //?? = LinuxCommands[3];
914         //?? = LinuxCommands[4];
915         //?? = LinuxCommands[5];
916         //?? = LinuxCommands[6];
917         //?? = LinuxCommands[7];
918         //?? = LinuxCommands[8];
919         //?? = LinuxCommands[9];
920         //?? = LinuxCommands[10];
921     }
922
923     if (NewCAMDataThreshold1 == 1) {
924         NewCAMDataThreshold1 = 0;
925         MaxAreaThreshold1 = fromCAMvaluesThreshold1[0];
926         MaxColThreshold1 = fromCAMvaluesThreshold1[1];
927         MaxRowThreshold1 = fromCAMvaluesThreshold1[2];
928
929         // using the calibration values and the line of best fit found from the distance of the green golf ball in relation to the real world, this equation calculates the distance the robot is
930         robotToBall1 = -0.00002154*MaxRowThreshold1*MaxRowThreshold1*MaxRowThreshold1 + 0.006486*MaxRowThreshold1*MaxRowThreshold1 - 0.6728*MaxRowThreshold1 + 25.42;
931
932         NextLargestAreaThreshold1 = fromCAMvaluesThreshold1[3];
933         NextLargestColThreshold1 = fromCAMvaluesThreshold1[4];
934         NextLargestRowThreshold1 = fromCAMvaluesThreshold1[5];
935
936         NextNextLargestAreaThreshold1 = fromCAMvaluesThreshold1[6];
937         NextNextLargestColThreshold1 = fromCAMvaluesThreshold1[7];
938         NextNextLargestRowThreshold1 = fromCAMvaluesThreshold1[8];
939         numThres1++;
940         if ((numThres1 % 5) == 0) {
941             // LED4 is GPIO97
942             GpioDataRegs.GPDT0GGLE.bit.GPIO97 = 1;
943         }
944     }
945
946     if (NewCAMDataThreshold2 == 1) {
947         NewCAMDataThreshold2 = 0;
948         MaxAreaThreshold2 = fromCAMvaluesThreshold2[0];
949         MaxColThreshold2 = fromCAMvaluesThreshold2[1];
950         MaxRowThreshold2 = fromCAMvaluesThreshold2[2];
951
952         // using the calibration values and the line of best fit found from the distance of the green golf ball in relation to the real world, this equation calculates the distance the robot is
953         if (ROBOTtps.y < 35){ //first hallway
954             robotToBall2 = -0.00002154*MaxRowThreshold2*MaxRowThreshold2*MaxRowThreshold2 + 0.006486*MaxRowThreshold2*MaxRowThreshold2 - 0.6728*MaxRowThreshold2 + 25.42;
955         }
956         else if (ROBOTtps.y >= 35){ //second hallway
957             if (MaxRowThreshold2 > 40)
958                 robotToBall2 = -0.00002154*MaxRowThreshold2*MaxRowThreshold2*MaxRowThreshold2 + 0.006486*MaxRowThreshold2*MaxRowThreshold2 - 0.6728*MaxRowThreshold2 + 25.42;
959             else if (MaxRowThreshold2 <= 40)
960                 robotToBall2 = 0;
961         }
962         NextLargestAreaThreshold2 = fromCAMvaluesThreshold2[3];
963         NextLargestColThreshold2 = fromCAMvaluesThreshold2[4];
964         NextLargestRowThreshold2 = fromCAMvaluesThreshold2[5];
965
966         NextNextLargestAreaThreshold2 = fromCAMvaluesThreshold2[6];
967         NextNextLargestColThreshold2 = fromCAMvaluesThreshold2[7];
968         NextNextLargestRowThreshold2 = fromCAMvaluesThreshold2[8];
969         numThres2++;
970         if ((numThres2 % 5) == 0) {
971             // LED5 is GPIO111
972             GpioDataRegs.GPDT0GGLE.bit.GPIO111 = 1;
973         }
974     }
975
976     if (NewCAMDataThreshold3 == 1) {
977         NewCAMDataThreshold3 = 0;
978         MaxAreaThreshold3 = fromCAMvaluesThreshold3[0];
979         MaxColThreshold3 = fromCAMvaluesThreshold3[1];
980         MaxRowThreshold3 = fromCAMvaluesThreshold3[2];
981
982         // using the calibration values and the line of best fit found from the distance of the green golf ball in relation to the real world, this equation calculates the distance the robot is
983         robotToBall3 = -0.00002154*MaxRowThreshold3*MaxRowThreshold3*MaxRowThreshold3 + 0.006486*MaxRowThreshold3*MaxRowThreshold3 - 0.6728*MaxRowThreshold3 + 25.42;
984
985         NextLargestAreaThreshold3 = fromCAMvaluesThreshold3[3];
986         NextLargestColThreshold3 = fromCAMvaluesThreshold3[4];
987         NextLargestRowThreshold3 = fromCAMvaluesThreshold3[5];
988
989         NextNextLargestAreaThreshold3 = fromCAMvaluesThreshold3[6];
990         NextNextLargestColThreshold3 = fromCAMvaluesThreshold3[7];
991         NextNextLargestRowThreshold3 = fromCAMvaluesThreshold3[8];
992         numThres3++;
993         if ((numThres3 % 5) == 0) {
994             // LED5 is GPIO111

```

```

993     GpioDataRegs.GPDT0GGLE.bit.GPI0111 = 1;
994 }
995 }
996
997 if (NewCAMDataAprilTag1 == 1) {
998     NewCAMDataAprilTag1 = 0;
999     tagx = fromCAMvaluesAprilTag1[0];
1000     tagy = fromCAMvaluesAprilTag1[1];
1001     tagz = fromCAMvaluesAprilTag1[2];
1002
1003     tagthetax = fromCAMvaluesAprilTag1[3];
1004     tagthetay = fromCAMvaluesAprilTag1[4];
1005     tagthetaz = fromCAMvaluesAprilTag1[5];
1006
1007     tagid = fromCAMvaluesAprilTag1[6];
1008
1009     numtag++;
1010     if ((numtag % 5) == 0) {
1011         // LED2 is GPIO94
1012         GpioDataRegs.GPCTOGGLE.bit.GPIO94 = 1;
1013     }
1014 }
1015
1016 if (NewLVData == 1) {
1017     NewLVData = 0;
1018     printLV1 = fromLVvalues[0];
1019     printLV2 = fromLVvalues[1];
1020     //?? = fromLVvalues[2];
1021     //?? = fromLVvalues[3];
1022     //?? = fromLVvalues[4];
1023     //?? = fromLVvalues[5];
1024     //?? = fromLVvalues[6];
1025     //?? = fromLVvalues[7];
1026 }
1027
1028
1029 if (new_optitrack == 1) {
1030     OPTITRACKps = UpdateOptitrackStates(ROBOTps, &newOPTITRACKpose);
1031     new_optitrack = 0;
1032 }
1033
1034 // Step 0: update B, u
1035 B[0][0] = cosf(ROBOTps.theta)*0.001;
1036 B[1][0] = sinf(ROBOTps.theta)*0.001;
1037 B[2][1] = 0.001;
1038 u[0][0] = 0.5*(LeftVel + RightVel);    // linear velocity of robot
1039 u[1][0] = gyrozz*(PI/180.0);    // angular velocity in rad/s (negative for right hand angle)
1040
1041 // Step 1: predict the state and estimate covariance
1042 Matrix3x2_Mult(B, u, Bu);    // Bu = B*u
1043 Matrix3x1_Add(x_pred, Bu, x_pred, 1.0, 1.0); // x_pred = x_pred(old) + Bu
1044 Matrix3x3_Add(P_pred, Q, P_pred, 1.0, 1.0); // P_pred = P_pred(old) + Q
1045 // Step 2: if there is a new measurement, then update the state
1046 if (1 == newOPTITRACKpose) {
1047     OptiNumUpdatesProcessed++; // checking how often OptiTrack is causing a Kalman update
1048     if ((OptiNumUpdatesProcessed%10)==0) {
1049         // LED 3
1050         GpioDataRegs.GPCTOGGLE.bit.GPIO95 = 1;
1051     }
1052     newOPTITRACKpose = 0;
1053     z[0][0] = OPTITRACKps.x;    // take in the Optitrack Motion Capture measurement
1054     z[1][0] = OPTITRACKps.y;
1055     // fix for OptiTrack problem at 180 degrees
1056     if (cosf(ROBOTps.theta) < -0.99) {
1057         z[2][0] = ROBOTps.theta;
1058     }
1059     else {
1060         z[2][0] = OPTITRACKps.theta;
1061     }
1062     // Step 2a: calculate the innovation/measurement residual, ytilde
1063     Matrix3x1_Add(z, x_pred, ytilde, 1.0, -1.0);    // ytilde = z-x_pred
1064     // Step 2b: calculate innovation covariance, S
1065     Matrix3x3_Add(P_pred, R, S, 1.0, 1.0);    // S = P_pred + R
1066     // Step 2c: calculate the optimal Kalman gain, K
1067     Matrix3x3_Invert(S, S_inv);
1068     Matrix3x3_Mult(P_pred, S_inv, K);    // K = P_pred*(S^-1)
1069     // Step 2d: update the state estimate x_pred = x_pred(old) + K*ytilde
1070     Matrix3x1_Mult(K, ytilde, temp_3x1);
1071     Matrix3x1_Add(x_pred, temp_3x1, x_pred, 1.0, 1.0);
1072     // Step 2e: update the covariance estimate P_pred = (I-K)*P_pred(old)
1073     Matrix3x3_Add(eye3, K, temp_3x3, 1.0, -1.0);
1074     Matrix3x3_Mult(temp_3x3, P_pred, P_pred);
1075 }    // end of correction step
1076
1077 // set ROBOTps to the updated and corrected Kalman values.
1078 ROBOTps.x = x_pred[0][0];
1079 ROBOTps.y = x_pred[1][0];
1080 ROBOTps.theta = x_pred[2][0];
1081
1082 // Make sure this function is called every time in this function even if you decide not to use its vref and turn
1083 // uses xy code to step through an array of positions
1084 if (xy_control(&vref, &turn, 1.0, ROBOTps.x, ROBOTps.y, robotdest[statePos].x, robotdest[statePos].y, ROBOTps.theta, 0.25, 0.5)) {
1085     statePos = (statePos+1)%NUMWAYPOINTS;
1086 }
1087
1088 dwell++;
1089 // state machine
1090 colcentroid1 = MaxColThreshold1 - 80;
1091 colcentroid2 = MaxColThreshold2 - 80; // 80 is used as the center column of the camera, so a centered green or orange ball centered gives centroid = 0 - DS
1092 colcentroid3 = MaxColThreshold3 - 80;
1093 switch (RobotState) {
1094 case 1: // command the robot to an X,Y point in the course - KL
1095     // vref and turn are the vref and turn returned from xy_control
1096     if (LADARfront < 1.2) {
1097         vref = 0.2;
1098         checkfronttally++;
1099         if (checkfronttally > 310) { // check if LADARfront < 1.2 for 310ms or 3 LADAR samples
1100             RobotState = 10; // Wall follow
1101             WallFollowtime = 0;
1102             right_wall_follow_state = 1;
1103         }
1104     } else {
1105         checkfronttally = 0;
1106     }
1107     // ensures that the robot does not move for 2 seconds after returning to state 1, so after is stops detecting a golf ball - ER
1108     statecount++;
1109     if (statecount<=2000){
1110         RobotState = 1;
1111     }else{
1112         if (MaxAreaThreshold1 > 30) { // green ball detection threshold chosen so camera noise (image disturbance, not literal) does not trigger chasing rolling ball behavior - DS
1113             RobotState = 20;
1114             statecount =0;
1115         }
1116         if (MaxAreaThreshold2 > 30) { // same detection threshold but for orange ball - DS
1117             RobotState = 30;
1118             statecount =0;
1119         }
1120         if (fabs(tagz) <= 350.468262) {
1121             if (tagid == 0.0){
1122                 RobotState = 40;
1123             }
1124             if (tagid == 1.0){
1125                 RobotState = 50;
1126             }
1127         }
1128     }
1129 }
1130 break;
1131 case 10: // right wall following if an obstacle gets in the way of robot - KL
1132     if (right_wall_follow_state == 1) {
1133         //Left Turn
1134         turn = Kp_front_wall*(14.5 - LADARfront);

```

```

1135     vref = front_turn_velocity;
1136     if (LADARfront > left_turn_Stop_threshold) {
1137         right_wall_follow_state = 2;
1138     }
1139 } else if (right_wall_follow_state == 2) {
1140     //Right Wall Follow
1141     turn = Kp_right_wal*(ref_right_wall - LADARrightfront);
1142     vref = foward_velocity;
1143     if (LADARfront < left_turn_Start_threshold) {
1144         right_wall_follow_state = 1;
1145     }
1146 }
1147 if (turn > turn_saturation) {
1148     turn = turn_saturation;
1149 }
1150 if (turn < -turn_saturation) {
1151     turn = -turn_saturation;
1152 }
1153
1154 WallFollowtime++;
1155 // exit wall following if at least 5 seconds have passed and front path is clear - KL
1156 if ( (WallFollowtime > 5000) && (LADARfront > 1.5) ) {
1157     RobotState = 1; //return to XY waypoint navigation - KL
1158     checkfronttally = 0;
1159 }
1160 break;
1161 // this state sets vref and turn so that the robot moves towards a detected green golf ball and after 1 second of waiting, sends the robot to state 22 which stops the robot - ER
case 20:
1162     // put vision code here
1163     if (MaxColThreshold1 == 0 || MaxAreaThreshold1 < 3){
1164         vref = 0;
1165         turn = 0;
1166     } else {
1167         vref = 0.75;
1168         turn = kpvision * (0 - colcentroid1);
1169         if (MaxRowThreshold1 > 100){ //KLEC: row number for rb to know how to close it is to the ball, need to tune the number for enough clearance so that gate doesn't push ball away
1170             RobotState = 22;
1171             count = 0;
1172         }
1173     }
1174 }
1175 break;
1176 case 22: // after no longer detecting the green golf ball, the robot stops moving - ER, lift arm and move tongue
1177     vref = 0;
1178     turn = 0;
1179
1180     count++;
1181     if (count>=1000){ //after 1 second, robot drives forward - KL
1182         RobotState = 24;
1183         count = 0;
1184     }
1185     break;
1186 case 24: // robot drives forward into the golf ball - KL
1187     vref = 0.5;
1188     turn = 0;
1189     setEPWM5B_RCServo(LTong); //KLEC: move the tongue to the left side, green balls go to the right compartment
1190     setEPWM6A_RCServo(Gate_0); //KLEC: open the gate
1191     count++;
1192     if (count>=2000){
1193         RobotState = 26;
1194         count = 0;
1195     }
1196     break;
1197 case 26: // robot pauses once ball is presumed to be captured - KL
1198     vref = 0;
1199     turn = 0;
1200     setEPWM6A_RCServo(Gate_C);
1201     setEPWM5B_RCServo(MTong);
1202     count++;
1203     if (count>=1000){ //after 1 second, robot returns to waypoint navigation - KL
1204         RobotState = 1;
1205         count = 0;
1206         statecount = 0;
1207     }
1208     break;
1209
1210 // states for orange ball
case 30:
1211     // put vision code here
1212     //saying that if the orange ball is not detected or doesn't fit in the area threshold, then to stop the robot - KP
1213     //this helps it not detect other orange objects in the room as the ball - KP
1214     if (MaxColThreshold2 == 0 || MaxAreaThreshold2 < 3){
1215         vref = 0;
1216         turn = 0;
1217     } else {
1218         //tells it what to do if it is detected, move to state 32 - KP
1219         vref = 0.75;
1220         turn = kpvision * (0 - colcentroid2);
1221         if (MaxRowThreshold2 > 100){
1222             RobotState = 32;
1223             count = 0;
1224         }
1225     }
1226 }
1227 break;
1228 case 32:
1229     //stopping and pausing with vref and turn at 0 - KP
1230     vref = 0;
1231     turn = 0;
1232     setEPWM5B_RCServo(RTong); //KLEC: move the tongue to the right side, orange balls go to the left compartment
1233     setEPWM6A_RCServo(Gate_0); //KLEC: open the gate
1234     count++;
1235     if (count>=1000){
1236         RobotState = 34;
1237         count = 0;
1238     }
1239     break;
1240 case 34:
1241     //telling robot to move forward then move to state 36 - KP
1242     vref = 0.5;
1243     turn = 0;
1244     setEPWM5B_RCServo(RTong);
1245     setEPWM6A_RCServo(Gate_0);
1246     count++;
1247     if (count>=1000){
1248         RobotState = 36;
1249         count = 0;
1250     }
1251     break;
1252 case 36:
1253     //another stop before it begins the route again - KP
1254     vref = 0;
1255     turn = 0;
1256     setEPWM6A_RCServo(Gate_C);
1257     setEPWM5B_RCServo(MTong);
1258     count++;
1259     if (count>=1000){
1260         RobotState = 1;
1261         count = 0;
1262         statecount = 0;
1263     }
1264     break;
1265 case 40://code telling the gate to open when it detects april tag 0
1266     vref = 0;
1267     turn = 0;
1268
1269     setEPWM6A_RCServo(Gate_0);
1270     setEPWM5B_RCServo(0);
1271
1272     if (dwell >= 2000){
1273         setEPWM6A_RCServo(Gate_C);
1274     }
1275
1276     count++;

```

```

1277     if (count>=1000){
1278         RobotState = 1;
1279         count = 0;
1280         statecount = 0;
1281     }
1282     break;
1283 case 50: //code telling the robot to turn
1284
1285     vref = 0;
1286     count++;
1287     if (count>=2000){
1288         RobotState = 52;
1289         count = 0;
1290     }
1291     break;
1292 case 52:
1293     current = ROBOTps.theta;
1294     target = 3 * HALFPI;
1295     turn = fabs(target - current);
1296     vref = 1;
1297     count++;
1298     if (count>=1000){
1299         RobotState = 1;
1300         count = 0;
1301         statecount = 0;
1302     }
1303     break;
1304
1305 default:
1306     break;
1307 }
1308
1309 //Must be called each time into this SWI1 function
1310 PIcontrl(&uLeft,&uRight,vref,turn,LeftWheel,RightWheel);
1311 // These below lines also must be called each time into this SWI1 function
1312 setEPWM1A(uLeft);
1313 setEPWM2A(-uRight);
1314 setF28027EPWM1A(uLeft);
1315 setF28027EPWM2A(-uRight);
1316 LeftWheel_1 = LeftWheel;
1317 RightWheel_1 = RightWheel;
1318
1319 if((timecount%250) == 0) {
1320     DataToLabView.floatData[0] = ROBOTps.x;
1321     DataToLabView.floatData[1] = ROBOTps.y;
1322     DataToLabView.floatData[2] = ROBOTps.theta;
1323     DataToLabView.floatData[3] = (float)timecount;
1324     DataToLabView.floatData[4] = 0.5*(LeftVel + RightVel);
1325     DataToLabView.floatData[5] = (float)RobotState;
1326     DataToLabView.floatData[6] = (float)statePos;
1327     DataToLabView.floatData[7] = LADARfront;
1328     LVsenddata[0] = '$'; // header for LVdata
1329     LVsenddata[1] = '\0';
1330     for (i=0;i<LVNUM_TOFROM_FLOATS*4;i++) {
1331         if (i%2==0) {
1332             LVsenddata[i+2] = DataToLabView.rawData[i/2] & 0xFF;
1333         } else {
1334             LVsenddata[i+2] = (DataToLabView.rawData[i/2]>>8) & 0xFF;
1335         }
1336     }
1337     serial_sendSCID(&SerialD, LVsenddata, 4*LVNUM_TOFROM_FLOATS + 2);
1338 }
1339
1340 }
1341 timecount++;
1342 if((timecount%200) == 0)
1343 {
1344     if(doneCal == 0) {
1345         GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1; // Blink Blue LED while calibrating
1346     }
1347     GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1; // Always Block Red LED
1348     UARTPrint = 1; // Tell While loop to print
1349 }
1350
1351 SpibRegs.SPIFFCT.bit.TXDLY = 16;
1352 CurrentChip = DAN28027;
1353 SpibRegs.SPIFFRX.bit.RXFIFL = 3;
1354 GpioDataRegs.GPACLEAR.bit.GPIO29 = 1;
1355 SpibRegs.SPITXBUF = 0x00DA;
1356 SpibRegs.SPITXBUF = EPwm1A_F28027;
1357 SpibRegs.SPITXBUF = EPwm2A_F28027;
1358
1359 //*****
1360 //
1361 // Restore registers saved:
1362 //
1363 DINT;
1364 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1365
1366 GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
1367 }
1368
1369 //
1370 // Connected to PIEIER12_10 (use MINT12 and MG12_10 masks):
1371 //
1372 __interrupt void SWI2_MiddlePriority(void) // RAM_CORRECTABLE_ERROR
1373 {
1374     // Set interrupt priority:
1375     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1376     IER |= MINT12;
1377     IER &= MINT12; // Set "global" priority
1378     PieCtrlRegs.PIEIER12.all &= MG12_10; // Set "group" priority
1379     PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
1380     asm(" NOP");
1381     EINT;
1382
1383 //*****
1384 // Insert SWI ISR Code here.....
1385 // LED1
1386 GpioDataRegs.GPATOGGLE.bit.GPIO22 = 1;
1387 if (LADARpingpong == 1) {
1388     // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1389     LADARrightfront = 19; // 19 is greater than max feet
1390     for (LADARi = 52; LADARi <= 56 ; LADARi++) {
1391         if (ladar_data[LADARi].distance_ping < LADARrightfront) {
1392             LADARrightfront = ladar_data[LADARi].distance_ping;
1393         }
1394     }
1395     // LADARfront is the min of dist 111, 112, 113, 114, 115
1396     LADARfront = 19;
1397     for (LADARi = 111; LADARi <= 115 ; LADARi++) {
1398         if (ladar_data[LADARi].distance_ping < LADARfront) {
1399             LADARfront = ladar_data[LADARi].distance_ping;
1400         }
1401     }
1402     LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta - PI/2.0));
1403     LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-LADARps.y*cosf(ROBOTps.theta - PI/2.0));
1404     for (LADARi = 0; LADARi < 228; LADARi++) {
1405         ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_ping*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1406         ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_ping*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1407     }
1408 } else if (LADARpingpong == 0) {
1409     // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1410     LADARrightfront = 19; // 19 is greater than max feet
1411     for (LADARi = 52; LADARi <= 56 ; LADARi++) {
1412         if (ladar_data[LADARi].distance_pong < LADARrightfront) {
1413             LADARrightfront = ladar_data[LADARi].distance_pong;
1414         }
1415     }
1416 }
1417 // LADARfront is the min of dist 111, 112, 113, 114, 115

```

```

1419     LADARfront = 19;
1420     for (LADARi = 111; LADARi <= 115 ; LADARi++) {
1421         if (ladar_data[LADARi].distance_pong < LADARfront) {
1422             LADARfront = ladar_data[LADARi].distance_pong;
1423         }
1424     }
1425     LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta - PI/2.0));
1426     LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-LADARps.y*cosf(ROBOTps.theta - PI/2.0));
1427     for (LADARi = 0; LADARi < 228; LADARi++) {
1428
1429         ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_pong*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1430         ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_pong*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1431
1432     }
1433 }
1434
1435
1436 //#####
1437 //
1438 // Restore registers saved:
1439 //
1440 DINT;
1441 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1442 }
1443 }
1444
1445 //
1446 // Connected to PIEIER12_11 (use MINT12 and MG12_11 masks):
1447 //
1448 __interrupt void SWI3_LowestPriority(void)    // FLASH_CORRECTABLE_ERROR
1449 {
1450     // Set interrupt priority:
1451     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1452     IER |= M_INT12;
1453     IER   &= MINT12;                // Set "global" priority
1454     PieCtrlRegs.PIEIER12.all &= MG12_11; // Set "group" priority
1455     PieCtrlRegs.PIEACK.all = 0xFFFF;    // Enable PIE interrupts
1456     __asm(" NOP");
1457     EINT;
1458     RCangle = readEncWheel(); //Lab RC servo exercise uses encoder angle as the command input for servo motion - DS
1459     // setEPWM3A_RCServo(RCangle); //RCangle is a value from -90 to 90
1460     // setEPWM3B_RCServo(RCangle); //RCangle is a value from -90 to 90
1461     // setEPWM5A_RCServo(RCangle); //RCangle is a value from -90 to 90 in use
1462     setEPWM5B_RCServo(RCangle); //RCangle is a value from -90 to 90 in use - Tongue
1463     setEPWM6A_RCServo(RCangle); //RCangle is a value from -90 to 90 in use - Gate
1464     //#####
1465     // Insert SWI ISR Code here.....
1466
1467     //#####
1468     //
1469     // Restore registers saved:
1470     //
1471     DINT;
1472     PieCtrlRegs.PIEIER12.all = TempPIEIER;
1473 }
1474 }
1475
1476 // ----- code for CAN start here -----
1477 __interrupt void can_isr(void)
1478 {
1479     int i = 0;
1480
1481     uint32_t status;
1482
1483     //
1484     // Read the CAN interrupt status to find the cause of the interrupt
1485     //
1486     status = CANGetInterruptCause(CANB_BASE);
1487
1488     //
1489     // If the cause is a controller status interrupt, then get the status
1490     //
1491     if(status == CAN_INT_INT0ID_STATUS)
1492     {
1493         //
1494         // Read the controller status. This will return a field of status
1495         // error bits that can indicate various errors. Error processing
1496         // is not done in this example for simplicity. Refer to the
1497         // API documentation for details about the error status bits.
1498         // The act of reading this status will clear the interrupt.
1499         //
1500         status = CANGetStatus(CANB_BASE);
1501     }
1502 }
1503
1504 //
1505 // Check if the cause is the receive message object 2
1506 //
1507 else if(status == RX_MSG_OBJ_ID_1)
1508 {
1509     //
1510     // Get the received message
1511     //
1512     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_1, rxMsgData);
1513
1514     for(i = 0; i<2; i++)
1515     {
1516         dis_raw_1[i] = rxMsgData[i];
1517     }
1518
1519     dis_1 = 256*dis_raw_1[1] + dis_raw_1[0];
1520
1521     measure_status_1 = rxMsgData[2];
1522
1523     //
1524     // Getting to this point means that the RX interrupt occurred on
1525     // message object 2, and the message RX is complete. Clear the
1526     // message object interrupt.
1527     //
1528     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_1);
1529
1530     //
1531     // Increment a counter to keep track of how many messages have been
1532     // received. In a real application this could be used to set flags to
1533     // indicate when a message is received.
1534     //
1535     rxMsgCount_1++;
1536
1537     //
1538     // Since the message was received, clear any error flags.
1539     //
1540     errorFlag = 0;
1541 }
1542
1543 else if(status == RX_MSG_OBJ_ID_2)
1544 {
1545     //
1546     // Get the received message
1547     //
1548     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_2, rxMsgData);
1549
1550     for(i = 0; i<2; i++)
1551     {
1552         dis_raw_2[i] = rxMsgData[i];
1553     }
1554
1555     dis_2 = 256*dis_raw_2[1] + dis_raw_2[0];
1556
1557     measure_status_2 = rxMsgData[2];
1558
1559     //
1560     // Getting to this point means that the RX interrupt occurred on

```

```

1561 // message object 2, and the message RX is complete. Clear the
1562 // message object interrupt.
1563 //
1564 CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_2);
1565
1566 //
1567 // Increment a counter to keep track of how many messages have been
1568 // received. In a real application this could be used to set flags to
1569 // indicate when a message is received.
1570 //
1571 rxMsgCount_2++;
1572
1573 //
1574 // Since the message was received, clear any error flags.
1575 //
1576 errorFlag = 0;
1577 }
1578
1579 else if(status == RX_MSG_OBJ_ID_3)
1580 {
1581 //
1582 // Get the received message
1583 //
1584 CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_3, rxMsgData);
1585
1586 for(i = 0; i<4; i++)
1587 {
1588     lightlevel_raw_1[i] = rxMsgData[i];
1589     quality_raw_1[i] = rxMsgData[i+4];
1590 }
1591
1592 lightlevel_1 = ((256.0*256.0*256.0)*lightlevel_raw_1[3] + (256.0*256.0)*lightlevel_raw_1[2] + 256.0*lightlevel_raw_1[1] + lightlevel_raw_1[0])/65535;
1593 quality_1 = ((256.0*256.0*256.0)*quality_raw_1[3] + (256.0*256.0)*quality_raw_1[2] + 256.0*quality_raw_1[1] + quality_raw_1[0])/65535;
1594
1595 //
1596 // Getting to this point means that the RX interrupt occurred on
1597 // message object 2, and the message RX is complete. Clear the
1598 // message object interrupt.
1599 //
1600 CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_3);
1601
1602 //
1603 // Since the message was received, clear any error flags.
1604 //
1605 errorFlag = 0;
1606 }
1607
1608
1609 else if(status == RX_MSG_OBJ_ID_4)
1610 {
1611 //
1612 // Get the received message
1613 //
1614 CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_4, rxMsgData);
1615
1616 for(i = 0; i<4; i++)
1617 {
1618     lightlevel_raw_2[i] = rxMsgData[i];
1619     quality_raw_2[i] = rxMsgData[i+4];
1620 }
1621
1622 lightlevel_2 = ((256.0*256.0*256.0)*lightlevel_raw_2[3] + (256.0*256.0)*lightlevel_raw_2[2] + 256.0*lightlevel_raw_2[1] + lightlevel_raw_2[0])/65535;
1623 quality_2 = ((256.0*256.0*256.0)*quality_raw_2[3] + (256.0*256.0)*quality_raw_2[2] + 256.0*quality_raw_2[1] + quality_raw_2[0])/65535;
1624
1625 //
1626 // Getting to this point means that the RX interrupt occurred on
1627 // message object 2, and the message RX is complete. Clear the
1628 // message object interrupt.
1629 //
1630 CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_4);
1631
1632 //
1633 // Since the message was received, clear any error flags.
1634 //
1635 errorFlag = 0;
1636 }
1637
1638 //
1639 // If something unexpected caused the interrupt, this would handle it.
1640 //
1641 else
1642 {
1643 //
1644 // Spurious interrupt handling can go here.
1645 //
1646 }
1647
1648 //
1649 // Clear the global interrupt flag for the CAN interrupt line
1650 //
1651 CANclearGlobalInterruptStatus(CANB_BASE, CAN_GLOBAL_INT_CANINT0);
1652
1653 //
1654 // Acknowledge this interrupt located in group 9
1655 //
1656 InterruptClearACKGroup(INTERRUPT_ACK_GROUP9);
1657 }
1658
1659 // ----- code for CAN end here -----
1660
1661

```

```

1 /* SERIAL.C: This code is designed to act as a low-level serial driver for
2    higher-level programming. Ideally, one could simply call init_serial()
3    to initialize the serial port, then use serial_send("data", 4) to send
4    an array of data (8-bit unsigned character strings).
5
6    WRITTEN BY : Paul Miller <pamiller@uiuc.edu>
7    $Id: serial.c,v 1.4 2003/08/08 16:08:56 paul Exp $
8 */
9 #include <stdio.h>
10 #include <stdlib.h>
11 #include <stdarg.h>
12 #include <string.h>
13 #include <math.h>
14 #include <limits.h>
15
16 #include "F28x_Project.h"    // Device Headerfile and Examples Include File
17 #include <buffer.h>
18 #include <F28379dSerial.h>
19 #include <F2837xD_sci.h>
20 #include "SE423Lib.h"
21
22 serialSCIA_t SerialA;
23 serialSCIB_t SerialB;
24 serialSCIC_t SerialC;
25 serialSCID_t SerialD;
26
27 char RXAdata = 0;
28 char RXBdata = 0;
29 char RXCdata = 0;
30 char RXDdata = 0;
31 uint32_t numRXA = 0;
32 uint32_t numRXB = 0;
33 uint32_t numRXC = 0;
34 uint32_t numRXD = 0;
35
36 char buf[477]; //store all the data
37 uint16_t dis_index = 0;
38 uint16_t G_count = 0; //number of thing received after 'G'
39 uint16_t LF_count = 0; //number of consecutive line feed
40 uint16_t buf_index = 0; //for debug
41 uint16_t get_dis_first; //if it is the first part of the two parts data
42 uint16_t print_dis_index=0;
43 uint32_t numRCXerrors = 0;
44 int16_t tempscid = 0;
45 int16_t distance = 0;
46 float checkdist = 0;
47
48 uint16_t COMA_state = 0;
49 uint16_t COMALSB = 0;
50
51 uint16_t received_CAM_countThreshold1 = 0;
52 uint16_t NewCAMDataThreshold1 = 0; // Flag new data
53 CAMRecFloats_t DataFromCameraThreshold1;
54 float fromCAMvaluesThreshold1[CAMNUM_FROM_FLOATS];
55
56 uint16_t received_CAM_countThreshold2 = 0;
57 uint16_t NewCAMDataThreshold2 = 0; // Flag new data
58 CAMRecFloats_t DataFromCameraThreshold2;
59 float fromCAMvaluesThreshold2[CAMNUM_FROM_FLOATS];
60
61 //EC: Declaration added for the third color
62 uint16_t received_CAM_countThreshold3 = 0;
63 uint16_t NewCAMDataThreshold3 = 0; // Flag new data
64 CAMRecFloats_t DataFromCameraThreshold3;
65 float fromCAMvaluesThreshold3[CAMNUM_FROM_FLOATS];
66
67 uint16_t received_CAM_countAprilTag1 = 0;
68 uint16_t NewCAMDataAprilTag1 = 0; // Flag new data
69 CAMRecFloats_t DataFromCameraAprilTag1;
70 float fromCAMvaluesAprilTag1[CAMNUM_FROM_FLOATS];
71
72 uint16_t tempLSB = 0;
73 LVRecFloats_t DataFromLabView;
74 LVSendFloats_t DataToLabView;
75 uint16_t received_LV_count = 0;
76 uint16_t NewLVData = 0;
77 char LVsenddata[LVNUM_TOFROM_FLOATS*4+2];
78 float fromLVvalues[LVNUM_TOFROM_FLOATS];
79
80 CMDRecFloats_t DataFromLinuxCMD;
81 uint16_t received_CMD_count = 0;
82 uint16_t newLinuxCommands = 0;
83 float LinuxCommands[CMDNUM_FROM_FLOATS];
84 uint16_t com_state = 0; //the state of receive from pi (SCID)
85 uint16_t LADARpingpong = 0;
86 uint16_t read_ladar_state = 0; //states for ladar reading, initialized as wait for S command
87 datapts_ladar_data[228]; //distance data from ladar
88

```

```

89 uint16_t new_optitrack = 0;
90 float Optitrackdata[OPTITRACKDATASIZE]; // x, y, heading, trackableID, framecount
91 uint16_t received_opti_count = 0; // the count for receiving optitrack data
92 optiData_t optirx;
93
94 char path_received[81];
95 int16_t received_path_count = 0;
96 int16_t newAstarPath = 0;
97 float ladar161distance = 0;
98 float ladar199distance = 0;
99
100
101 // for SerialA
102 uint16_t init_serialSCIA(serialSCIA_t *s, uint32_t baud)
103 {
104     volatile struct SCI_REGS *sci;
105     uint32_t clk;
106
107     if (s == &SerialA) {
108         sci = &SciaRegs;
109         s->sci = sci;
110
111         init_bufferSCIA(&s->TX);
112
113         GPIO_SetupPinMux(43, GPIO_MUX_CPU1, 0);
114         GPIO_SetupPinOptions(43, GPIO_INPUT, GPIO_PULLUP);
115         GPIO_SetupPinMux(42, GPIO_MUX_CPU1, 0);
116         GPIO_SetupPinOptions(42, GPIO_OUTPUT, GPIO_PUSHPULL);
117
118     } else {
119         return 1;
120     }
121
122     /* init for standard baud, 8N1 comm */
123     sci->SCICTL1.bit.SWRESET = 0; // init SCI state machines and opt flags
124     sci->SCICCR.all = 0x0;
125     sci->SCICTL1.all = 0x0;
126     sci->SCICTL2.all = 0x0;
127     sci->SCIPRI.all = 0x0;
128     clk = LSPCLK_HZ; // set baud rate
129     clk /= baud*8;
130     clk--;
131     sci->SCILBAUD.all = clk & 0xFF;
132     sci->SCIHBAUD.all = (clk >> 8) & 0xFF;
133
134     sci->SCICCR.bit.SCICCHAR = 0x7; // (8) 8 bits per character
135     sci->SCICCR.bit.PARIYENA = 0; // (N) disable parity calculation
136     sci->SCICCR.bit.STOPBITS = 0; // (1) transmit 1 stop bit
137     sci->SCICCR.bit.LOOPBKENA = 0; // disable loopback test
138     sci->SCICCR.bit.ADDRIDLE_MODE = 0; // idle-line mode (non-multiprocessor SCI comm)
139
140     sci->SCIFFCT.bit.FFTXDLY = 0; // TX: zero-delay
141
142     sci->SCIFFTX.bit.SCIFFENA = 1; // enable SCI fifo enhancements
143     sci->SCIFFTX.bit.TXFIFORESET = 0;
144     sci->SCIFFTX.bit.TXFFIL = 0x0; // TX: fifo interrupt at all levels ??? is this correct
145     sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
146     sci->SCIFFTX.bit.TXFFIENA = 0; // TX: disable fifo interrupt
147     sci->SCIFFTX.bit.TXFIFORESET = 1;
148
149     sci->SCIFFRX.bit.RXFIFORESET = 0; // RX: fifo reset
150     sci->SCIFFRX.bit.RXFFINTCLR = 1; // RX: clear interrupt flag
151     sci->SCIFFRX.bit.RXFFIENA = 1; // RX: enable fifo interrupt
152     sci->SCIFFRX.bit.RXFFIL = 0x1; // RX: fifo interrupt
153     sci->SCIFFRX.bit.RXFIFORESET = 1; // RX: re-enable fifo
154
155     sci->SCICTL2.bit.RXBKINTENA = 0; // disable receiver/error interrupt
156     sci->SCICTL2.bit.TXINTENA = 0; // disable transmitter interrupt
157
158     sci->SCICTL1.bit.TXWAKE = 0;
159     sci->SCICTL1.bit.SLEEP = 0; // disable sleep mode
160     sci->SCICTL1.bit.RXENA = 1; // enable SCI receiver
161     sci->SCICTL1.bit.RXERRINTENA = 0; // disable receive error interrupt
162     sci->SCICTL1.bit.TXENA = 1; // enable SCI transmitter
163     sci->SCICTL1.bit.SWRESET = 1; // re-enable SCI
164
165     /* enable PIE interrupts */
166     if (s == &SerialA) {
167         PieCtrlRegs.PIEIER9.bit.INTx1 = 1;
168         PieCtrlRegs.PIEIER9.bit.INTx2 = 1;
169         IER |= (M_INT9);
170         PieCtrlRegs.PIEACK.all = (PIEACK_GROUP9);
171         GPIO_SetupPinMux(43, GPIO_MUX_CPU1, 15);
172         GPIO_SetupPinOptions(43, GPIO_INPUT, GPIO_PULLUP);
173         GPIO_SetupPinMux(42, GPIO_MUX_CPU1, 15);
174         GPIO_SetupPinOptions(42, GPIO_OUTPUT, GPIO_PUSHPULL);
175
176     }
177
178     return 0;
179 }

```

```

180
181 void uninit_serialSCIA(serialSCIA_t *s)
182 {
183     volatile struct SCI_REGS *sci = s->sci;
184
185     /* disable PIE interrupts */
186     if (s == &SerialA) {
187         PieCtrlRegs.PIEIER9.bit.INTx1 = 0;
188         PieCtrlRegs.PIEIER9.bit.INTx2 = 0;
189         IER &= ~M_INT9;
190     }
191     sci->SCICTL1.bit.RXERRINTENA = 0; // disable receive error interrupt
192     sci->SCICTL2.bit.RXBKINTENA = 0; // disable receiver/error interrupt
193     sci->SCICTL2.bit.TXINTENA = 0; // disable transmitter interrupt
194
195     sci->SCICTL1.bit.RXENA = 0; // disable SCI receiver
196     sci->SCICTL1.bit.TXENA = 0; // disable SCI transmitter
197 }
198
199
200
201 /*****
202  * SERIAL_SEND()
203  *
204  * "User level" function to send data via serial. Return value is the
205  * length of data successfully copied to the TX buffer.
206  *****/
207 uint16_t serial_sendSCIA(serialSCIA_t *s, char *data, uint16_t len)
208 {
209     uint16_t i = 0;
210     if (len && s->TX.size < BUF_SIZESCIA) {
211         for (i = 0; i < len; i++) {
212             if (buf_writeSCIA_1(&s->TX, data[i] & 0x00FF) != 0) break;
213         }
214         s->sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
215         s->sci->SCIFFTX.bit.TXFFIENA = 1; // TX: enable fifo interrupt
216     }
217     return i;
218 }
219
220 // For SerialB
221 uint16_t init_serialSCIB(serialSCIB_t *s, uint32_t baud)
222 {
223     volatile struct SCI_REGS *sci;
224     uint32_t clk;
225
226     if (s == &SerialB) {
227         sci = &ScibRegs;
228         s->sci = sci;
229
230         init_bufferSCIB(&s->TX);
231
232         GPIO_SetupPinMux(15, GPIO_MUX_CPU1, 0);
233         GPIO_SetupPinOptions(15, GPIO_INPUT, GPIO_PULLUP);
234         GPIO_SetupPinMux(14, GPIO_MUX_CPU1, 0);
235         GPIO_SetupPinOptions(14, GPIO_OUTPUT, GPIO_PUSHPULL);
236
237     } else {
238         return 1;
239     }
240 }
241
242 /* init for standard baud,8N1 comm */
243 sci->SCICTL1.bit.SWRESET = 0; // init SCI state machines and opt flags
244 sci->SCICCR.all = 0x0;
245 sci->SCICTL1.all = 0x0;
246 sci->SCICTL2.all = 0x0;
247 sci->SCIPRI.all = 0x0;
248 clk = LSPCLK_HZ; // set baud rate
249 clk /= baud*8;
250 clk--;
251 sci->SCILBAUD.all = clk & 0xFF;
252 sci->SCIHBAUD.all = (clk >> 8) & 0xFF;
253
254 sci->SCICCR.bit.SCICCHAR = 0x7; // (8) 8 bits per character
255 sci->SCICCR.bit.PARIYENA = 0; // (N) disable parity calculation
256 sci->SCICCR.bit.STOPBITS = 0; // (1) transmit 1 stop bit
257 sci->SCICCR.bit.LOOPBKENA = 0; // disable loopback test
258 sci->SCICCR.bit.ADDRIDLE_MODE = 0; // idle-line mode (non-multiprocessor SCI comm)
259
260 sci->SCIFFCT.bit.FFTXDLY = 0; // TX: zero-delay
261
262 sci->SCIFFTX.bit.SCIFFENA = 1; // enable SCI fifo enhancements
263 sci->SCIFFTX.bit.TXFIFORESET = 0;
264 sci->SCIFFTX.bit.TXFFIL = 0x0; // TX: fifo interrupt at all levels ??? is this correct
265 sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
266 sci->SCIFFTX.bit.TXFFIENA = 0; // TX: disable fifo interrupt
267 sci->SCIFFTX.bit.TXFIFORESET = 1;
268
269 sci->SCIFFRX.bit.RXFIFORESET = 0; // RX: fifo reset
270 sci->SCIFFRX.bit.RXFFINTCLR = 1; // RX: clear interrupt flag

```

```

271 sci->SCIFFRX.bit.RXFFIENA = 1; // RX: enable fifo interrupt
272 sci->SCIFFRX.bit.RXFFIL = 0x1; // RX: fifo interrupt
273 sci->SCIFFRX.bit.RXFIFORESET = 1; // RX: re-enable fifo
274
275 sci->SCICTL2.bit.RXBKINTENA = 0; // disable receiver/error interrupt
276 sci->SCICTL2.bit.TXINTENA = 0; // disable transmitter interrupt
277
278 sci->SCICTL1.bit.TXWAKE = 0;
279 sci->SCICTL1.bit.SLEEP = 0; // disable sleep mode
280 sci->SCICTL1.bit.RXENA = 1; // enable SCI receiver
281 sci->SCICTL1.bit.RXERRINTENA = 0; // disable receive error interrupt
282 sci->SCICTL1.bit.TXENA = 1; // enable SCI transmitter
283 sci->SCICTL1.bit.SWRESET = 1; // re-enable SCI
284
285 /* enable PIE interrupts */
286 if (s == &SerialB) {
287     PieCtrlRegs.PIEIER9.bit.INTx3 = 1;
288     PieCtrlRegs.PIEIER9.bit.INTx4 = 1;
289     IER |= (M_INT9);
290     PieCtrlRegs.PIEACK.all = (PIEACK_GROUP9);
291     GPIO_SetupPinMux(15, GPIO_MUX_CPU1, 2);
292     GPIO_SetupPinOptions(15, GPIO_INPUT, GPIO_PULLUP);
293     GPIO_SetupPinMux(14, GPIO_MUX_CPU1, 2);
294     GPIO_SetupPinOptions(14, GPIO_OUTPUT, GPIO_PUSHPULL);
295 }
296
297
298 return 0;
299 }
300
301 void uninit_serialSCIB(serialSCIB_t *s)
302 {
303     volatile struct SCI_REGS *sci = s->sci;
304
305     /* disable PIE interrupts */
306     if (s == &SerialB) {
307         PieCtrlRegs.PIEIER9.bit.INTx3 = 0;
308         PieCtrlRegs.PIEIER9.bit.INTx4 = 0;
309         IER &= ~M_INT9;
310     }
311
312     sci->SCICTL1.bit.RXERRINTENA = 0; // disable receive error interrupt
313     sci->SCICTL2.bit.RXBKINTENA = 0; // disable receiver/error interrupt
314     sci->SCICTL2.bit.TXINTENA = 0; // disable transmitter interrupt
315
316     sci->SCICTL1.bit.RXENA = 0; // disable SCI receiver
317     sci->SCICTL1.bit.TXENA = 0; // disable SCI transmitter
318 }
319
320
321
322 /*****
323  * SERIAL_SEND()
324  *
325  * "User level" function to send data via serial. Return value is the
326  * length of data successfully copied to the TX buffer.
327  *****/
328 uint16_t serial_sendSCIB(serialSCIB_t *s, char *data, uint16_t len)
329 {
330     uint16_t i = 0;
331     if (len && s->TX.size < BUF_SIZE_SCIB) {
332         for (i = 0; i < len; i++) {
333             if (buf_writeSCIB_1(&s->TX, data[i] & 0x00FF) != 0) break;
334         }
335         s->sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
336         s->sci->SCIFFTX.bit.TXFFIENA = 1; // TX: enable fifo interrupt
337     }
338     return i;
339 }
340
341 // for SerialC
342 uint16_t init_serialSCIC(serialSCIC_t *s, uint32_t baud)
343 {
344     volatile struct SCI_REGS *sci;
345     uint32_t clk;
346
347     if (s == &SerialC) {
348         sci = &ScicRegs;
349         s->sci = sci;
350
351         init_bufferSCIC(&s->TX);
352         GPIO_SetupPinMux(139, GPIO_MUX_CPU1, 0);
353         GPIO_SetupPinOptions(139, GPIO_INPUT, GPIO_PULLUP);
354         GPIO_SetupPinMux(56, GPIO_MUX_CPU1, 0);
355         GPIO_SetupPinOptions(56, GPIO_OUTPUT, GPIO_PUSHPULL);
356
357     } else {
358         return 1;
359     }
360
361     /* init for standard baud, 8N1 comm */

```

```

362 sci->SCICTL1.bit.SWRESET = 0;          // init SCI state machines and opt flags
363 sci->SCICCR.all = 0x0;
364 sci->SCICTL1.all = 0x0;
365 sci->SCICTL2.all = 0x0;
366 sci->SCIPRI.all = 0x0;
367 clk = LSPCLK_HZ;                      // set baud rate
368 clk /= baud*8;
369 clk--;
370 sci->SCILBAUD.all = clk & 0xFF;
371 sci->SCIHBAUD.all = (clk >> 8) & 0xFF;
372
373 sci->SCICCR.bit.SCICHAR = 0x7;          // (8) 8 bits per character
374 sci->SCICCR.bit.PARITYENA = 0;          // (N) disable parity calculation
375 sci->SCICCR.bit.STOPBITS = 0;           // (1) transmit 1 stop bit
376 sci->SCICCR.bit.LOOPBKENA = 0;          // disable loopback test
377 sci->SCICCR.bit.ADDRIDLE_MODE = 0;      // idle-line mode (non-multiprocessor SCI comm)
378
379 sci->SCIFFCT.bit.FFTXDLY = 0;           // TX: zero-delay
380
381 sci->SCIFFTX.bit.SCIFFENA = 1;           // enable SCI fifo enhancements
382 sci->SCIFFTX.bit.TXFIFORESET = 0;
383 sci->SCIFFTX.bit.TXFFIL = 0x0; // TX: fifo interrupt at all levels      ??? is this correct
384 sci->SCIFFTX.bit.TXFFINTCLR = 1;        // TX: clear interrupt flag
385 sci->SCIFFTX.bit.TXFFIENA = 0;          // TX: disable fifo interrupt
386 sci->SCIFFTX.bit.TXFIFORESET = 1;
387
388 sci->SCIFFRX.bit.RXFIFORESET = 0;        // RX: fifo reset
389 sci->SCIFFRX.bit.RXFFINTCLR = 1;        // RX: clear interrupt flag
390 sci->SCIFFRX.bit.RXFFIENA = 1;          // RX: enable fifo interrupt
391 sci->SCIFFRX.bit.RXFFIL = 0x1;          // RX: fifo interrupt
392 sci->SCIFFRX.bit.RXFIFORESET = 1;        // RX: re-enable fifo
393
394 sci->SCICTL2.bit.RXBKINTENA = 0;          // disable receiver/error interrupt
395 sci->SCICTL2.bit.TXINTENA = 0;          // disable transmitter interrupt
396
397 sci->SCICTL1.bit.TXWAKE = 0;
398 sci->SCICTL1.bit.SLEEP = 0;             // disable sleep mode
399 sci->SCICTL1.bit.RXENA = 1;             // enable SCI receiver
400 sci->SCICTL1.bit.RXERRINTENA = 0;        // disable receive error interrupt
401 sci->SCICTL1.bit.TXENA = 1;             // enable SCI transmitter
402 sci->SCICTL1.bit.SWRESET = 1;           // re-enable SCI
403
404 /* enable PIE interrupts */
405 if (s == &SerialC) {
406     PieCtrlRegs.PIEIER8.bit.INTx5 = 1;
407     PieCtrlRegs.PIEIER8.bit.INTx6 = 1;
408     PieCtrlRegs.PIEACK.all = (PIEACK_GROUP8);
409     IER |= (M_INT8);
410     GPIO_SetupPinMux(139, GPIO_MUX_CPU1, 6);
411     GPIO_SetupPinOptions(139, GPIO_INPUT, GPIO_PULLUP);
412     GPIO_SetupPinMux(56, GPIO_MUX_CPU1, 6);
413     GPIO_SetupPinOptions(56, GPIO_OUTPUT, GPIO_PUSHPULL);
414 }
415
416 return 0;
417 }
418
419 void uninit_serialSCIC(serialSCIC_t *s)
420 {
421     volatile struct SCI_REGS *sci = s->sci;
422
423     /* disable PIE interrupts */
424     if (s == &SerialC) {
425         PieCtrlRegs.PIEIER8.bit.INTx5 = 0;
426         PieCtrlRegs.PIEIER8.bit.INTx6 = 0;
427         IER &= ~M_INT8;
428     }
429
430     sci->SCICTL1.bit.RXERRINTENA = 0;      // disable receive error interrupt
431     sci->SCICTL2.bit.RXBKINTENA = 0;      // disable receiver/error interrupt
432     sci->SCICTL2.bit.TXINTENA = 0;        // disable transmitter interrupt
433
434     sci->SCICTL1.bit.RXENA = 0;           // disable SCI receiver
435     sci->SCICTL1.bit.TXENA = 0;           // disable SCI transmitter
436 }
437
438
439
440 /*****
441  * SERIAL_SEND()
442  *
443  * "User level" function to send data via serial. Return value is the
444  * length of data successfully copied to the TX buffer.
445  *****/
446 uint16_t serial_sendSCIC(serialSCIC_t *s, char *data, uint16_t len)
447 {
448     uint16_t i = 0;
449     if (len && s->TX.size < BUF_SIZE_SCIC) {
450         for (i = 0; i < len; i++) {
451             if (buf_writeSCIC_1(&s->TX, data[i] & 0x00FF) != 0) break;
452         }
453     }

```

```

453     s->sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
454     s->sci->SCIFFTX.bit.TXFFIENA = 1;    // TX: enable fifo interrupt
455 }
456 return i;
457 }
458
459 // For Serial D
460 uint16_t init_serialSCID(serialSCID_t *s, uint32_t baud)
461 {
462     volatile struct SCI_REGS *sci;
463     uint32_t clk;
464
465     if (s == &SerialD) {
466         sci = &ScidRegs;
467         s->sci = sci;
468
469         init_bufferSCID(&s->TX);
470         GPIO_SetupPinMux(105, GPIO_MUX_CPU1, 0);
471         GPIO_SetupPinOptions(105, GPIO_INPUT, GPIO_PULLUP);
472         GPIO_SetupPinMux(104, GPIO_MUX_CPU1, 0);
473         GPIO_SetupPinOptions(104, GPIO_OUTPUT, GPIO_PUSHPULL);
474     }
475     else {
476         return 1;
477     }
478
479     /* init for standard baud,8N1 comm */
480     sci->SCICTL1.bit.SWRESET = 0; // init SCI state machines and opt flags
481     sci->SCICCR.all = 0x0;
482     sci->SCICTL1.all = 0x0;
483     sci->SCICTL2.all = 0x0;
484     sci->SCIPRI.all = 0x0;
485     clk = LSPCLK_HZ; // set baud rate
486     clk /= baud*8;
487     clk--;
488     sci->SCILBAUD.all = clk & 0xFF;
489     sci->SCIHBAUD.all = (clk >> 8) & 0xFF;
490
491     sci->SCICCR.bit.SCICCHAR = 0x7; // (8) 8 bits per character
492     sci->SCICCR.bit.PARITYENA = 0; // (N) disable parity calculation
493     sci->SCICCR.bit.STOPBITS = 0; // (1) transmit 1 stop bit
494     sci->SCICCR.bit.LOOPBKENA = 0; // disable loopback test
495     sci->SCICCR.bit.ADDRIDLE_MODE = 0; // idle-line mode (non-multiprocessor SCI comm)
496
497     sci->SCIFFCT.bit.FFTXDLY = 0; // TX: zero-delay
498
499     sci->SCIFFTX.bit.SCIFFENA = 1; // enable SCI fifo enhancements
500     sci->SCIFFTX.bit.TXFIFORESET = 0;
501     sci->SCIFFTX.bit.TXFFIL = 0x0; // TX: fifo interrupt at all levels ??? is this correct
502     sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
503     sci->SCIFFTX.bit.TXFFIENA = 0; // TX: disable fifo interrupt
504     sci->SCIFFTX.bit.TXFIFORESET = 1;
505
506     sci->SCIFFRX.bit.RXFIFORESET = 0; // RX: fifo reset
507     sci->SCIFFRX.bit.RXFFINTCLR = 1; // RX: clear interrupt flag
508     sci->SCIFFRX.bit.RXFFIENA = 1; // RX: enable fifo interrupt
509     sci->SCIFFRX.bit.RXFFIL = 0x1; // RX: fifo interrupt
510     sci->SCIFFRX.bit.RXFIFORESET = 1; // RX: re-enable fifo
511
512     sci->SCICTL2.bit.RXBKINTENA = 0; // disable receiver/error interrupt
513     sci->SCICTL2.bit.TXINTENA = 0; // disable transmitter interrupt
514
515     sci->SCICTL1.bit.TXWAKE = 0;
516     sci->SCICTL1.bit.SLEEP = 0; // disable sleep mode
517     sci->SCICTL1.bit.RXENA = 1; // enable SCI receiver
518     sci->SCICTL1.bit.RXERRINTENA = 0; // disable receive error interrupt
519     sci->SCICTL1.bit.TXENA = 1; // enable SCI transmitter
520     sci->SCICTL1.bit.SWRESET = 1; // re-enable SCI
521
522     /* enable PIE interrupts */
523     if (s == &SerialD) {
524         PieCtrlRegs.PIEIER8.bit.INTx7 = 1;
525         PieCtrlRegs.PIEIER8.bit.INTx8 = 1;
526         PieCtrlRegs.PIEACK.all = (PIEACK_GROUP8);
527         IER |= (M_INT8);
528         GPIO_SetupPinMux(105, GPIO_MUX_CPU1, 6);
529         GPIO_SetupPinOptions(105, GPIO_INPUT, GPIO_PULLUP);
530         GPIO_SetupPinMux(104, GPIO_MUX_CPU1, 6);
531         GPIO_SetupPinOptions(104, GPIO_OUTPUT, GPIO_PUSHPULL);
532     }
533 }
534
535 return 0;
536 }
537
538 void uninit_serialSCID(serialSCID_t *s)
539 {
540     volatile struct SCI_REGS *sci = s->sci;
541
542     /* disable PIE interrupts */
543     if (s == &SerialD) {

```

```

544     PieCtrlRegs.PIEIER8.bit.INTx7 = 0;
545     PieCtrlRegs.PIEIER8.bit.INTx8 = 0;
546     IER &= ~M_INT8;
547 }
548
549 sci->SCICTL1.bit.RXERRINTENA = 0; // disable receive error interrupt
550 sci->SCICTL2.bit.RXBKINTENA = 0; // disable receiver/error interrupt
551 sci->SCICTL2.bit.TXINTENA = 0; // disable transmitter interrupt
552
553 sci->SCICTL1.bit.RXENA = 0; // disable SCI receiver
554 sci->SCICTL1.bit.TXENA = 0; // disable SCI transmitter
555 }
556
557
558
559 /*****
560 * SERIAL_SEND()
561 *
562 * "User level" function to send data via serial. Return value is the
563 * length of data successfully copied to the TX buffer.
564 *****/
565 uint16_t serial_sendSCID(serialSCID_t *s, char *data, uint16_t len)
566 {
567     uint16_t i = 0;
568     if (len && s->TX.size < BUF_SIZE_SCID) {
569         for (i = 0; i < len; i++) {
570             if (buf_writeSCID_1(&s->TX, data[i] & 0x00FF) != 0) break;
571         }
572         s->sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
573         s->sci->SCIFFTX.bit.TXFFIENA = 1; // TX: enable fifo interrupt
574     }
575     return i;
576 }
577
578
579
580 /*****
581 * TXxINT_DATA_SENT()
582 *
583 * Executed when transmission is ready for additional data. These functions
584 * read the next char of data and put it in the TXBUF register for transfer.
585 *****/
586 #ifdef _FLASH
587 #pragma CODE_SECTION(TXAINT_data_sent, ".TI.ramfunc");
588 #endif
589 __interrupt void TXAINT_data_sent(void)
590 {
591     char data;
592     if (buf_readSCIA_1(&SerialA.TX, 0, &data) == 0) {
593         while ( (buf_readSCIA_1(&SerialA.TX, 0, &data) == 0)
594             && (SerialA.sci->SCIFFTX.bit.TXFFST != 0x10) ) {
595             buf_removeSCIA(&SerialA.TX, 1);
596             SerialA.sci->SCITXBUF.all = data;
597         }
598     } else {
599         SerialA.sci->SCIFFTX.bit.TXFFIENA = 0; // TX: disable fifo interrupt
600     }
601     SerialA.sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
602     PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;
603 }
604
605
606 //for serialB
607 #ifdef _FLASH
608 #pragma CODE_SECTION(TXBINT_data_sent, ".TI.ramfunc");
609 #endif
610 __interrupt void TXBINT_data_sent(void)
611 {
612     char data;
613     if (buf_readSCIB_1(&SerialB.TX, 0, &data) == 0) {
614         while ( (buf_readSCIB_1(&SerialB.TX, 0, &data) == 0)
615             && (SerialB.sci->SCIFFTX.bit.TXFFST != 0x10) ) {
616             buf_removeSCIB(&SerialB.TX, 1);
617             SerialB.sci->SCITXBUF.all = data;
618         }
619     } else {
620         SerialB.sci->SCIFFTX.bit.TXFFIENA = 0; // TX: disable fifo interrupt
621     }
622     SerialB.sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
623     PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;
624 }
625
626
627 //for serialC
628 #ifdef _FLASH
629 #pragma CODE_SECTION(TXCINT_data_sent, ".TI.ramfunc");
630 #endif
631 __interrupt void TXCINT_data_sent(void)
632 {
633     char data;
634     if (buf_readSCIC_1(&SerialC.TX, 0, &data) == 0) {

```

```

635     while ( (buf_readSCIC_1(&SerialC.TX,0,&data) == 0)
636             && (SerialC.sci->SCIFFTX.bit.TXFFST != 0x10) ) {
637         buf_removeSCIC(&SerialC.TX, 1);
638         SerialC.sci->SCITXBUF.all = data;
639     }
640 } else {
641     SerialC.sci->SCIFFTX.bit.TXFFIENA = 0;      // TX: disable fifo interrupt
642 }
643 SerialC.sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
644 PieCtrlRegs.PIEACK.all = PIEACK_GROUP8;
645 }
646
647 //for serialD
648 #ifdef _FLASH
649 #pragma CODE_SECTION(TXDINT_data_sent, ".TI.ramfunc");
650 #endif
651 __interrupt void TXDINT_data_sent(void)
652 {
653     char data;
654     if (buf_readSCID_1(&SerialD.TX,0,&data) == 0) {
655         while ( (buf_readSCID_1(&SerialD.TX,0,&data) == 0)
656                 && (SerialD.sci->SCIFFTX.bit.TXFFST != 0x10) ) {
657             buf_removeSCID(&SerialD.TX, 1);
658             SerialD.sci->SCITXBUF.all = data;
659         }
660     } else {
661         SerialD.sci->SCIFFTX.bit.TXFFIENA = 0;      // TX: disable fifo interrupt
662     }
663     SerialD.sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
664     PieCtrlRegs.PIEACK.all = PIEACK_GROUP8;
665 }
666
667 //for SerialA
668 #ifdef _FLASH
669 #pragma CODE_SECTION(RXAINT_recv_ready, ".TI.ramfunc");
670 #endif
671 __interrupt void RXAINT_recv_ready(void)
672 {
673     RXAdata = SciaRegs.SCIRXBUF.all;
674
675     /* SCI PE or FE error */
676     if (RXAdata & 0xC000) {
677         SciaRegs.SCICTL1.bit.SWRESET = 0;
678         SciaRegs.SCICTL1.bit.SWRESET = 1;
679         SciaRegs.SCIFFRX.bit.RXFIFORESET = 0;
680         SciaRegs.SCIFFRX.bit.RXFIFORESET = 1;
681     } else {
682         RXAdata = RXAdata & 0x00FF;
683         if (COMA_state == 0) {
684
685             if (RXAdata == '*') {
686                 COMA_state = 1; //for receiving path
687             } else {
688                 COMA_state = 0;
689             }
690         } else if (COMA_state == 1) {
691
692             if (RXAdata == '*') {
693                 COMA_state = 10; // into the read mode
694                 received_CAM_countThreshold1 = 0;
695             } else if (RXAdata == '!') {
696                 COMA_state = 20; // into the read mode
697                 received_CAM_countThreshold2 = 0;
698             } else if (RXAdata == '$') {
699                 COMA_state = 30; // into the read mode
700                 received_CAM_countAprilTag1 = 0;
701             } else if (RXAdata == '&') {
702                 COMA_state = 40; //EC: into the read mode
703                 received_CAM_countThreshold3 = 0;
704             }
705             else {
706                 COMA_state = 0;
707             }
708         } else if (COMA_state == 10) {
709
710             if ((received_CAM_countThreshold1 % 2) == 0) {
711                 COMALSB = RXAdata;
712             } else {
713                 DataFromCameraThreshold1.rawData[received_CAM_countThreshold1/2] = (RXAdata << 8) | COMALSB;
714             }
715             received_CAM_countThreshold1++;
716             if (received_CAM_countThreshold1 >= 4*CAMNUM_FROM_FLOATS) {
717                 received_CAM_countThreshold1 = 0;
718                 COMA_state = 0;
719                 fromCAMvaluesThreshold1[0] = DataFromCameraThreshold1.floatData[0];
720                 fromCAMvaluesThreshold1[1] = DataFromCameraThreshold1.floatData[1];
721                 fromCAMvaluesThreshold1[2] = DataFromCameraThreshold1.floatData[2];
722                 fromCAMvaluesThreshold1[3] = DataFromCameraThreshold1.floatData[3];
723                 fromCAMvaluesThreshold1[4] = DataFromCameraThreshold1.floatData[4];
724                 fromCAMvaluesThreshold1[5] = DataFromCameraThreshold1.floatData[5];
725                 fromCAMvaluesThreshold1[6] = DataFromCameraThreshold1.floatData[6];

```

```

726         fromCAMvaluesThreshold1[7] = DataFromCameraThreshold1.floatData[7];
727         fromCAMvaluesThreshold1[8] = DataFromCameraThreshold1.floatData[8];
728         NewCAMDataThreshold1 = 1; // Flag new data
729     }
730 } else if (COMA_state == 20) {
731     if ((received_CAM_countThreshold2 % 2) == 0) {
732         COMALSB = RXAdata;
733     } else {
734         DataFromCameraThreshold2.rawData[received_CAM_countThreshold2/2] = (RXAdata << 8) | COMALSB;
735     }
736     received_CAM_countThreshold2++;
737     if (received_CAM_countThreshold2 >= 4*CAMNUM_FROM_FLOATS) {
738         received_CAM_countThreshold2 = 0;
739         COMA_state = 0;
740         fromCAMvaluesThreshold2[0] = DataFromCameraThreshold2.floatData[0];
741         fromCAMvaluesThreshold2[1] = DataFromCameraThreshold2.floatData[1];
742         fromCAMvaluesThreshold2[2] = DataFromCameraThreshold2.floatData[2];
743         fromCAMvaluesThreshold2[3] = DataFromCameraThreshold2.floatData[3];
744         fromCAMvaluesThreshold2[4] = DataFromCameraThreshold2.floatData[4];
745         fromCAMvaluesThreshold2[5] = DataFromCameraThreshold2.floatData[5];
746         fromCAMvaluesThreshold2[6] = DataFromCameraThreshold2.floatData[6];
747         fromCAMvaluesThreshold2[7] = DataFromCameraThreshold2.floatData[7];
748         fromCAMvaluesThreshold2[8] = DataFromCameraThreshold2.floatData[8];
749         NewCAMDataThreshold2 = 1; // Flag new data
750     }
751 } else if (COMA_state == 30) {
752     if ((received_CAM_countAprilTag1 % 2) == 0) {
753         COMALSB = RXAdata;
754     } else {
755         DataFromCameraAprilTag1.rawData[received_CAM_countAprilTag1/2] = (RXAdata << 8) | COMALSB;
756     }
757     received_CAM_countAprilTag1++;
758     if (received_CAM_countAprilTag1 >= 4*CAMNUM_FROM_FLOATS) {
759         received_CAM_countAprilTag1 = 0;
760         COMA_state = 0;
761         fromCAMvaluesAprilTag1[0] = DataFromCameraAprilTag1.floatData[0];
762         fromCAMvaluesAprilTag1[1] = DataFromCameraAprilTag1.floatData[1];
763         fromCAMvaluesAprilTag1[2] = DataFromCameraAprilTag1.floatData[2];
764         fromCAMvaluesAprilTag1[3] = DataFromCameraAprilTag1.floatData[3];
765         fromCAMvaluesAprilTag1[4] = DataFromCameraAprilTag1.floatData[4];
766         fromCAMvaluesAprilTag1[5] = DataFromCameraAprilTag1.floatData[5];
767         fromCAMvaluesAprilTag1[6] = DataFromCameraAprilTag1.floatData[6];
768         fromCAMvaluesAprilTag1[7] = DataFromCameraAprilTag1.floatData[7];
769         fromCAMvaluesAprilTag1[8] = DataFromCameraAprilTag1.floatData[8];
770         NewCAMDataAprilTag1 = 1; // Flag new data
771     }
772 } else if (COMA_state == 40) {
773     if ((received_CAM_countThreshold3 % 2) == 0) {
774         COMALSB = RXAdata;
775     } else {
776         DataFromCameraThreshold3.rawData[received_CAM_countThreshold3/2] = (RXAdata << 8) | COMALSB;
777     }
778     received_CAM_countThreshold3++;
779     if (received_CAM_countThreshold3 >= 4*CAMNUM_FROM_FLOATS) {
780         received_CAM_countThreshold3 = 0;
781         COMA_state = 0;
782         fromCAMvaluesThreshold3[0] = DataFromCameraThreshold3.floatData[0];
783         fromCAMvaluesThreshold3[1] = DataFromCameraThreshold3.floatData[1];
784         fromCAMvaluesThreshold3[2] = DataFromCameraThreshold3.floatData[2];
785         fromCAMvaluesThreshold3[3] = DataFromCameraThreshold3.floatData[3];
786         fromCAMvaluesThreshold3[4] = DataFromCameraThreshold3.floatData[4];
787         fromCAMvaluesThreshold3[5] = DataFromCameraThreshold3.floatData[5];
788         fromCAMvaluesThreshold3[6] = DataFromCameraThreshold3.floatData[6];
789         fromCAMvaluesThreshold3[7] = DataFromCameraThreshold3.floatData[7];
790         fromCAMvaluesThreshold3[8] = DataFromCameraThreshold3.floatData[8];
791         NewCAMDataThreshold3 = 1; // Flag new data
792     }
793 }
794
795     numRXA ++;
796 }
797
798 SciaRegs.SCIFFRX.bit.RXFFINTCLR = 1;
799 PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;
800
801 }
802
803 //for SerialB
804 #ifdef _FLASH
805 #pragma CODE_SECTION(RXBINT_recv_ready, ".TI.ramfunc");
806 #endif
807 __interrupt void RXBINT_recv_ready(void)
808 {
809     RXBdata = ScibRegs.SCIRXBUF.all;
810
811     /* SCI PE or FE error */
812     if (RXBdata & 0xC000) {
813         ScibRegs.SCICTL1.bit.SWRESET = 0;
814         ScibRegs.SCICTL1.bit.SWRESET = 1;
815         SciaRegs.SCIFFRX.bit.RXFIFORESET = 0;
816         SciaRegs.SCIFFRX.bit.RXFIFORESET = 1;

```

```

817     } else {
818         RXBdata = RXBdata & 0x00FF;
819         // Do something with recieved character
820         numRXB ++;
821     }
822
823     ScibRegs.SCIFFRX.bit.RXFFINTCLR = 1;
824     PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;
825 }
826
827
828 // for SerialC
829 #ifdef _FLASH
830 #pragma CODE_SECTION(RXCINT_recv_ready, ".TI.ramfunc");
831 #endif
832 __interrupt void RXCINT_recv_ready(void)
833 {
834     RXCdata = ScicRegs.SCIRXBUF.all;
835
836     /* SCI PE or FE error */
837     if (RXCdata & 0xC000) {
838         ScicRegs.SCICTL1.bit.SWRESET = 0;
839         ScicRegs.SCICTL1.bit.SWRESET = 1;
840         ScicRegs.SCIFFRX.bit.RXFIFORESET = 0;
841         ScicRegs.SCIFFRX.bit.RXFIFORESET = 1;
842     } else {
843         RXCdata = RXCdata & 0x00FF;
844         if (read_ladar_state == 0) { //wait for the starting char 'G'
845             buf_index = 0;
846             if (RXCdata == 'G') {
847                 read_ladar_state = 1;
848                 G_count = 1;
849             }
850         } else if (read_ladar_state == 1) { //get the headers
851             G_count += 1; //count the number of headers
852             //check the status code and if it is not '0', wait for the next packet
853             if (G_count == 11) {
854                 if (RXCdata != '0') {
855                     read_ladar_state = 0;
856                 }
857             }
858             if (G_count >= 12) { //there are total 12 things before the data
859                 read_ladar_state = 2;
860                 get_dis_first = 1;
861                 dis_index = 0;
862             }
863         } else if (read_ladar_state == 2) { //get actual data
864             if (RXCdata == 0x0a) { //check for line feed
865                 LF_count += 1;
866             } else {
867                 LF_count = 0;
868                 if (get_dis_first == 1) {
869                     tempscid = (RXCdata - 0x30) << 6; //first data is the top 6 bit of the distance reading
870                     get_dis_first = 0;
871                 } else if (get_dis_first == 0) {
872                     distance = tempscid + (RXCdata - 0x30); //second data is the bottom 6 bit of the distance reading
873                     if (distance == 0 || distance == 6) { //error code
874                         distance = LADAR_MAX_READING; //set to the max reading
875                     } else if (distance == 16) { //distance is 4096mm
876                         distance = 4096;
877                     }
878                     if (LADARpingpong == 0) {
879                         //ladar_data[dis_index%228].distance_ping = distance*0.00328084; // fill in the distance_ping array
880                         checkdist = distance*0.00328084;
881                         if ((checkdist > 0.25) && (checkdist < 20.0)) {
882                             ladar_data[dis_index].distance_ping = checkdist; // fill in the distance_ping array
883                             if (dis_index == 161) {
884                                 ladar161distance = checkdist;
885                             }
886                             if (dis_index == 199) {
887                                 ladar199distance = checkdist;
888                             }
889                         } else {
890                             numRCXerrors++;
891                             ladar_data[dis_index].distance_ping = ladar_data[dis_index].distance_pong;
892                             if (dis_index == 161) {
893                                 ladar161distance = ladar_data[dis_index].distance_pong;
894                             }
895                             if (dis_index == 199) {
896                                 ladar199distance = ladar_data[dis_index].distance_pong;
897                             }
898                         }
899                     }
900                 } else if (LADARpingpong == 1) {
901                     //ladar_data[dis_index%228].distance_pong = distance*0.00328084; // fill in the distance_pong array
902                     checkdist = distance*0.00328084;
903                     if ((checkdist >= 0.25) && (checkdist < 20.0)) {
904                         ladar_data[dis_index].distance_pong = checkdist; // fill in the distance_ping array
905                         if (dis_index == 161) {
906                             ladar161distance = checkdist;
907                         }
908                     }
909                 }
910             }
911         }
912     }
913 }

```

```

908         if (dis_index == 199) {
909             ladar199distance = checkdist;
910         }
911     } else {
912         numRCXerrors++;
913         ladar_data[dis_index].distance_ping = ladar_data[dis_index].distance_ping;
914         if (dis_index == 161) {
915             ladar161distance = ladar_data[dis_index].distance_ping;
916         }
917         if (dis_index == 199) {
918             ladar199distance = ladar_data[dis_index].distance_ping;
919         }
920     }
921 }
922 dis_index += 1;
923 get_dis_first = 1;
924 }
925 }
926 if (LF_count == 2) { //two consecutive line feed means the end of the current data stream
927     read_ladar_state = 0;
928     print_dis_index = dis_index;
929     PostSWI2(); // Manually cause the interrupt for the SWI2
930     LADARpingpong = 1-LADARpingpong;
931 }
932 }
933 buf[buf_index % 477] = RXCdata;//fill in the buf array, debug purpose
934 buf_index++;
935 numRXC ++;
936 }
937 }
938
939 ScicRegs.SCIFFRX.bit.RXFFINTCLR = 1;
940 PieCtrlRegs.PIEACK.all = PIEACK_GROUP8;
941 }
942
943
944 //for SerialD
945 #ifdef _FLASH
946 #pragma CODE_SECTION(RXDINT_recv_ready, ".TI.ramfunc");
947 #endif
948 __interrupt void RXDINT_recv_ready(void)
949 {
950     uint16_t i = 0;//for loop
951     RXDdata = ScidRegs.SCIRXBUF.all;
952
953     /* SCI PE or FE error */
954     if (RXDdata & 0xC000) {
955         ScidRegs.SCICTL1.bit.SWRESET = 0;
956         ScidRegs.SCICTL1.bit.SWRESET = 1;
957         ScidRegs.SCIFFRX.bit.RXFIFORESET = 0;
958         ScidRegs.SCIFFRX.bit.RXFIFORESET = 1;
959     } else {
960         RXDdata = RXDdata & 0x00FF;
961         if (com_state == 0) {
962             if (RXDdata == '*') {
963                 com_state = 1; //for receiving path
964             } else if (RXDdata == '0') {
965                 com_state = 2; //for receiving optitrack data
966             } else {
967                 com_state = 0;
968             }
969         } else if (com_state == 1) {
970             if (RXDdata == '*') {
971                 com_state = 10; // into the read mode
972                 received_path_count = 0;
973                 newAstarPath = 0;
974             } else if (RXDdata == '$') {
975                 com_state = 30;
976                 received_LV_count = 0;
977             } else if (RXDdata == '!') {
978                 com_state = 40;
979                 received_CMD_count = 0;
980             } else {
981                 com_state = 0;
982             }
983         } else if (com_state == 10) { // this should not occurred left it here for future code with A* path planning
984             path_received[received_path_count] = RXDdata;
985             received_path_count++;
986             if (received_path_count >= 81) {
987                 received_path_count = 0;
988                 com_state = 0;
989                 newAstarPath = 1;
990             }
991         } else if (com_state == 2) {
992             if (RXDdata == 'p'){
993                 com_state = 20;
994             } else {
995                 com_state = 0;
996             }
997         } else if (com_state == 20) {
998             if (RXDdata == 't'){

```

```

999         com_state = 21;
1000     } else {
1001         com_state = 0;
1002     }
1003 } else if (com_state == 21) {
1004     if (RXDdata == 'i'){
1005         com_state = 22;
1006     } else {
1007         com_state = 0;
1008     }
1009 } else if (com_state == 22) {
1010     if (RXDdata == '.'){
1011         com_state = 23;
1012         // received_opti_count = 0;
1013     } else {
1014         com_state = 0;
1015     }
1016 } else if (com_state == 23) {
1017     if (received_opti_count % 2 == 0){ // this is for motion tracking system left for future code
1018         tempLSB = RXDdata;
1019     } else {
1020         optirx.rawData[received_opti_count/2] = (RXDdata << 8)|tempLSB;
1021     }
1022     received_opti_count++;
1023     if (received_opti_count >= OPTITRACKDATASIZE*4) {
1024         if (new_optitrack == 0) {
1025             for (i=0;i<OPTITRACKDATASIZE;i++) { // x, y, heading, trackableID, framecount
1026                 Optitrackdata[i] = optirx.floatData[i];
1027             }
1028             new_optitrack = 1;
1029         }
1030         received_opti_count = 0;
1031         com_state = 0;
1032     }
1033 } else if (com_state == 30) {
1034     if ((received_LV_count % 2) == 0) {
1035         tempLSB = RXDdata;
1036     } else {
1037         DataFromLabView.rawData[received_LV_count/2] = (RXDdata << 8)|tempLSB;
1038     }
1039     received_LV_count++;
1040     if (received_LV_count >= 4*LVNUM_TOFROM_FLOATS) {
1041         received_LV_count = 0;
1042         com_state = 0;
1043         fromLVvalues[0] = DataFromLabView.floatData[0];
1044         fromLVvalues[1] = DataFromLabView.floatData[1];
1045         fromLVvalues[2] = DataFromLabView.floatData[2];
1046         fromLVvalues[3] = DataFromLabView.floatData[3];
1047         fromLVvalues[4] = DataFromLabView.floatData[4];
1048         fromLVvalues[5] = DataFromLabView.floatData[5];
1049         fromLVvalues[6] = DataFromLabView.floatData[6];
1050         fromLVvalues[7] = DataFromLabView.floatData[7];
1051         NewLVData = 1; // Flag new data
1052     }
1053 } else if (com_state == 40) {
1054     if ((received_CMD_count % 2) == 0) {
1055         tempLSB = RXDdata;
1056     } else {
1057         DataFromLinuxCMD.rawData[received_CMD_count/2] = (RXDdata << 8)|tempLSB;
1058     }
1059     received_CMD_count++;
1060     if (received_CMD_count >= 4*CMDNUM_FROM_FLOATS) {
1061         received_CMD_count = 0;
1062         com_state = 0;
1063
1064         LinuxCommands[0] = DataFromLinuxCMD.floatData[0];
1065         LinuxCommands[1] = DataFromLinuxCMD.floatData[1];
1066         LinuxCommands[2] = DataFromLinuxCMD.floatData[2];
1067         LinuxCommands[3] = DataFromLinuxCMD.floatData[3];
1068         LinuxCommands[4] = DataFromLinuxCMD.floatData[4];
1069         LinuxCommands[5] = DataFromLinuxCMD.floatData[5];
1070         LinuxCommands[6] = DataFromLinuxCMD.floatData[6];
1071         LinuxCommands[7] = DataFromLinuxCMD.floatData[7];
1072         LinuxCommands[8] = DataFromLinuxCMD.floatData[8];
1073         LinuxCommands[9] = DataFromLinuxCMD.floatData[9];
1074         LinuxCommands[10] = DataFromLinuxCMD.floatData[10];
1075         newLinuxCommands = 1; // Flag new data
1076     }
1077 }
1078 }
1079 }
1080 numRXD ++;
1081 ScidRegs.SCIFFRX.bit.RXFFINTCLR = 1;
1082 PieCtrlRegs.PIEACK.all = PIEACK_GROUP8;
1083 }
1084
1085
1086 // SerialA only setup for Tera Term connection
1087 char serial_printf_bufSCIA[BUF_SIZE_SCIA];
1088
1089 uint16_t serial_printf(serialSCIA_t *s, char *fmt, ...)

```

```

1090 {
1091     va_list ap;
1092
1093     va_start(ap,fmt);
1094     vsprintf(serial_printf_bufSCIA,fmt,ap);
1095     va_end(ap);
1096
1097     return serial_sendSCIA(s,serial_printf_bufSCIA,strlen(serial_printf_bufSCIA));
1098 }
1099
1100 //For Text LCD SCIB only
1101 char UART_printf_bufferSCIB[BUF_SIZEESCIB];
1102 void UART_vprintfLine(unsigned char line, char *format, va_list ap)
1103 {
1104     char sendmsg[24];
1105
1106     int i;
1107
1108     vsprintf(UART_printf_bufferSCIB,format,ap);
1109
1110     // Add header information and pad end of transfer with spaces to clear display
1111     sendmsg[0] = 0xFE;
1112     sendmsg[1] = 'G';
1113     sendmsg[2] = 1;
1114     sendmsg[3] = line;
1115     for (i=4;i<24;i++) {
1116         if (i >= strlen(UART_printf_bufferSCIB)+4) {
1117             sendmsg[i] = ' ';
1118         } else {
1119             sendmsg[i] = UART_printf_bufferSCIB[i-4];
1120         }
1121     }
1122     serial_sendSCIB(&SerialB,sendmsg,24);
1123 }
1124
1125 void UART_printfLine(unsigned char line, char *format, ...)
1126 {
1127     va_list ap;
1128     va_start(ap, format);
1129     UART_vprintfLine(line,format,ap);
1130 }

```